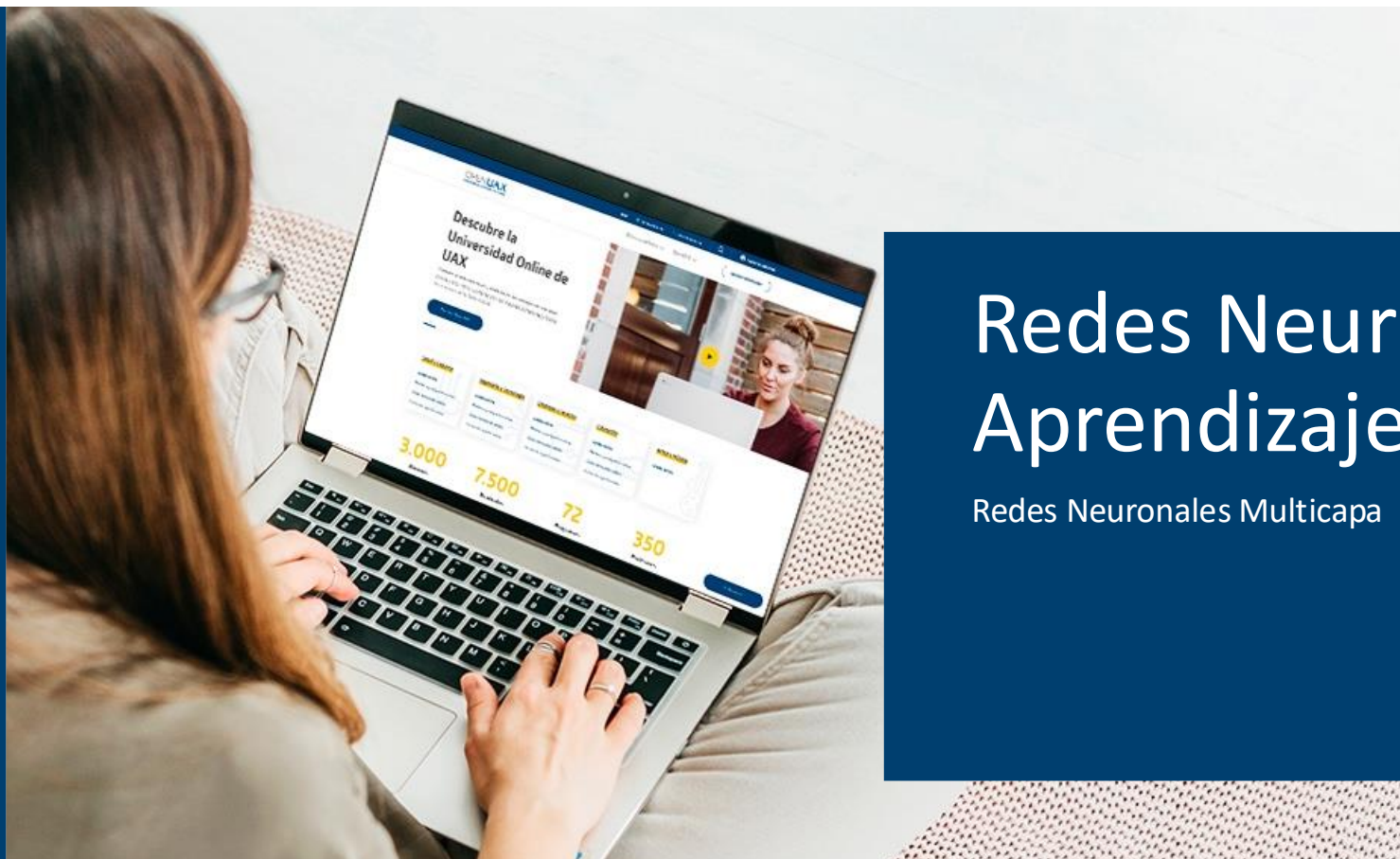


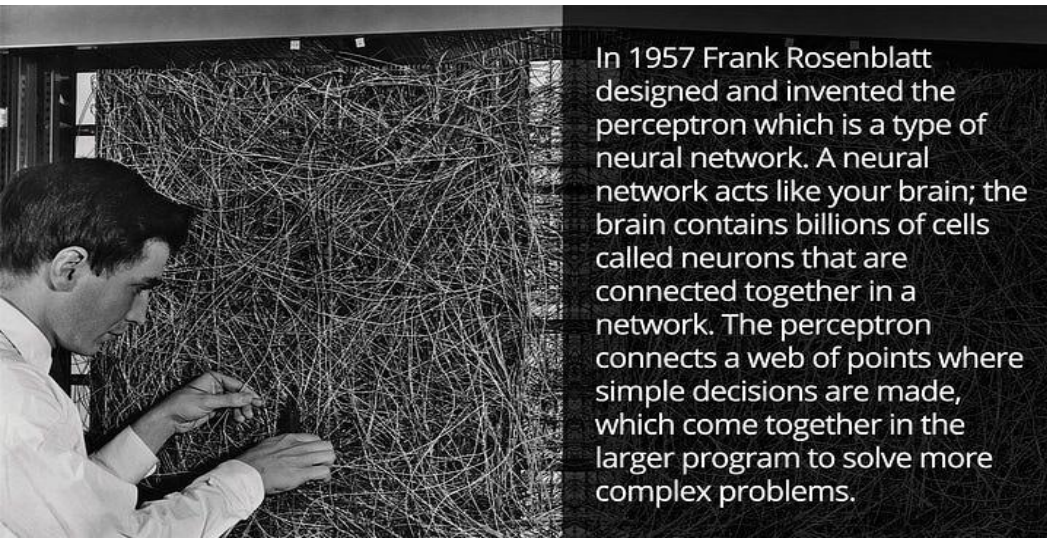
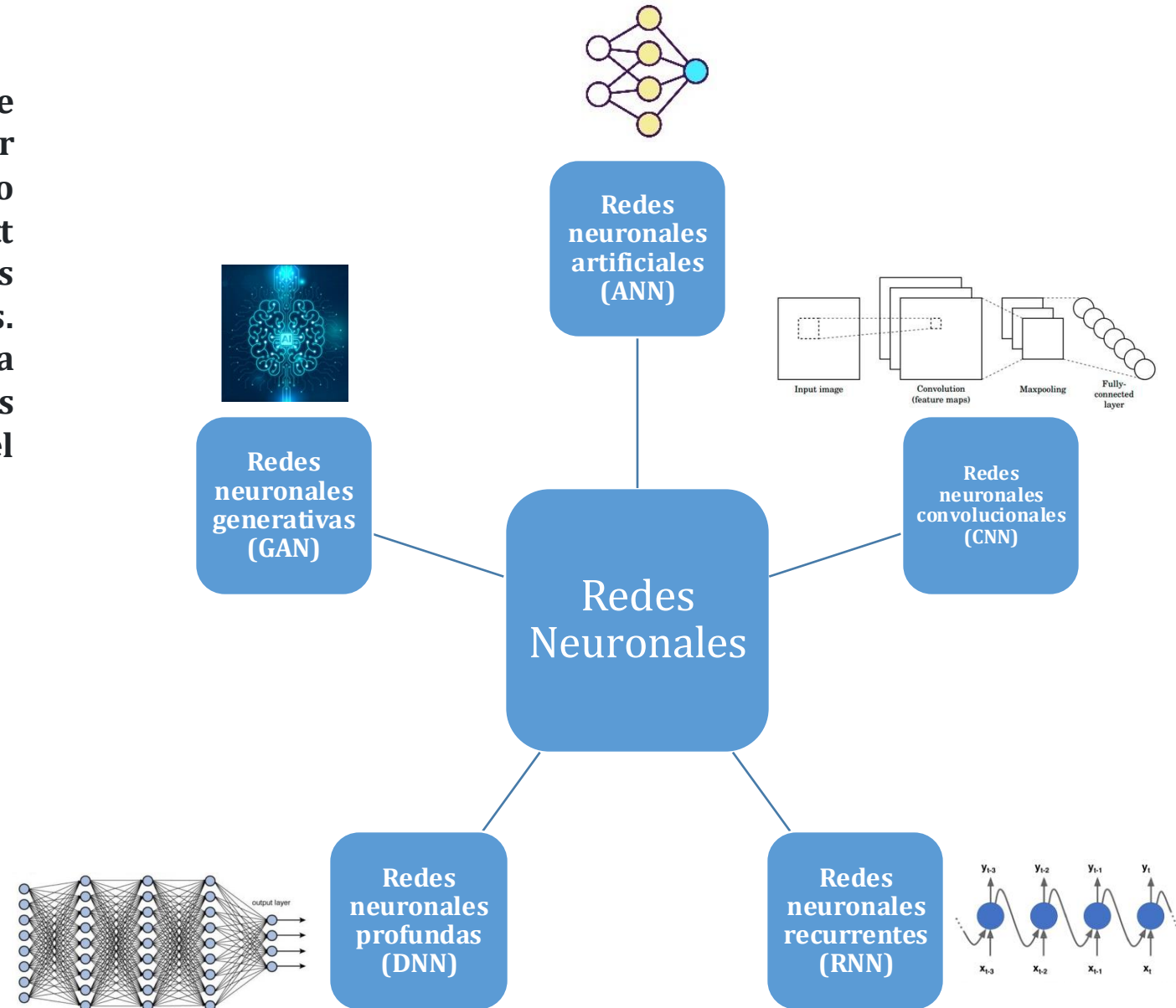


Redes Neuronales y Aprendizaje Profundo

Redes Neuronales Multicapa

El concepto de red neuronal se remonta a la década de 1940, cuando McCulloch y Pitts propusieron el primer modelo precursor de neurona artificial. Además, no fue hasta la década de los 50' cuando Rosenblatt introdujo el perceptrón, que tuvo aplicaciones limitadas en el reconocimiento de patrones simples. Sin embargo, la verdadera revolución ocurrió en la década de 1980 con la introducción de los algoritmos de retropropagación, que permitieron el entrenamiento eficiente.

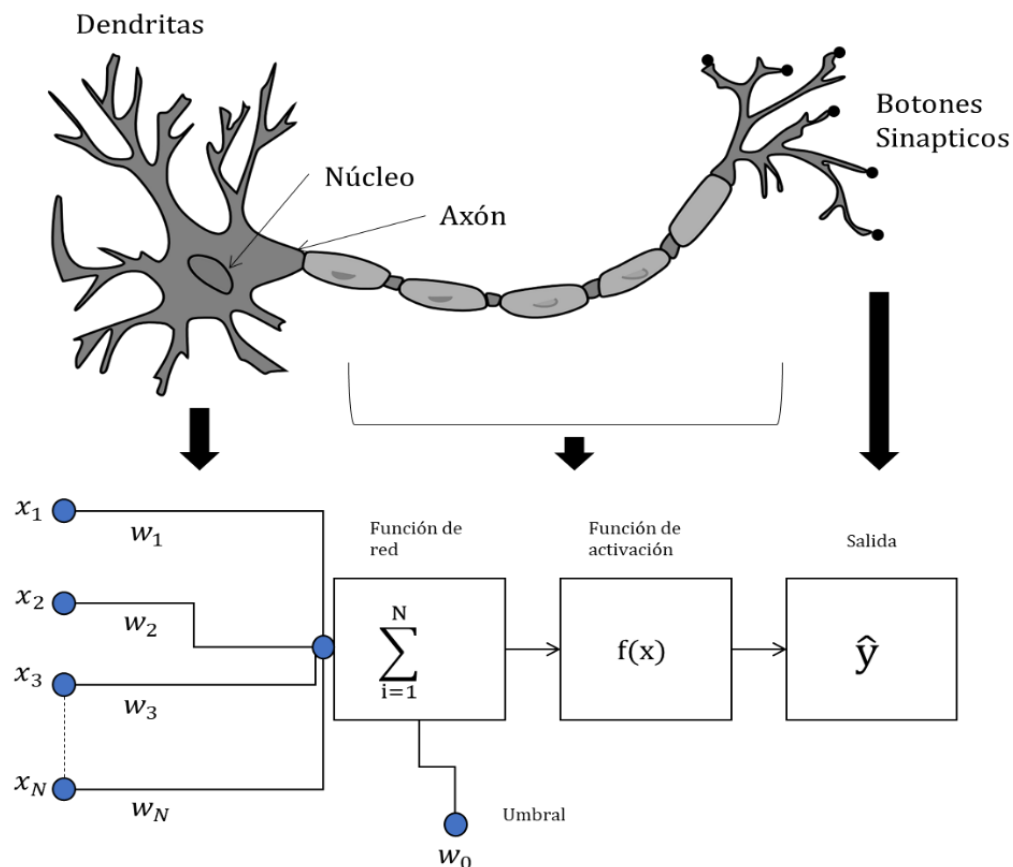
Redes neuronales



In 1957 Frank Rosenblatt designed and invented the perceptron which is a type of neural network. A neural network acts like your brain; the brain contains billions of cells called neurons that are connected together in a network. The perceptron connects a web of points where simple decisions are made, which come together in the larger program to solve more complex problems.

Redes neuronales

Las **Redes Neuronales Humanas** consisten en miles de millones de neuronas biológicas conectadas por sinapsis, procesando información sensorial y realizando tareas complejas mediante patrones de activación y plasticidad sináptica. Aprenden de forma continua y no supervisada, adaptándose a nuevas situaciones. En cambio, las **Redes Neuronales Artificiales** están compuestas por nodos y conexiones ponderadas, procesan información mediante cálculos matemáticos, y aprenden ajustando pesos a través de entrenamiento supervisado.



En una red neuronal, **la entrada representa** los datos iniciales, como imágenes, texto o números, que alimentan el modelo. Los pesos son parámetros que determinan la influencia de cada entrada en las neuronas de la red. **La preactivación es el resultado de combinar las entradas con sus pesos y sumar un sesgo**, que permite ajustar la activación sin depender solo de los datos. Luego, se aplica **una función de activación**, que introduce no linealidad y ayuda a la red a aprender patrones complejos. La activación resultante se propaga a la siguiente capa, repitiendo el proceso hasta llegar a la salida, que representa la predicción del modelo. **Durante el entrenamiento, la red ajusta los pesos y sesgos para minimizar el error.** Este ajuste se realiza mediante **descenso de gradiente**, optimizando el rendimiento de la red con cada iteración.

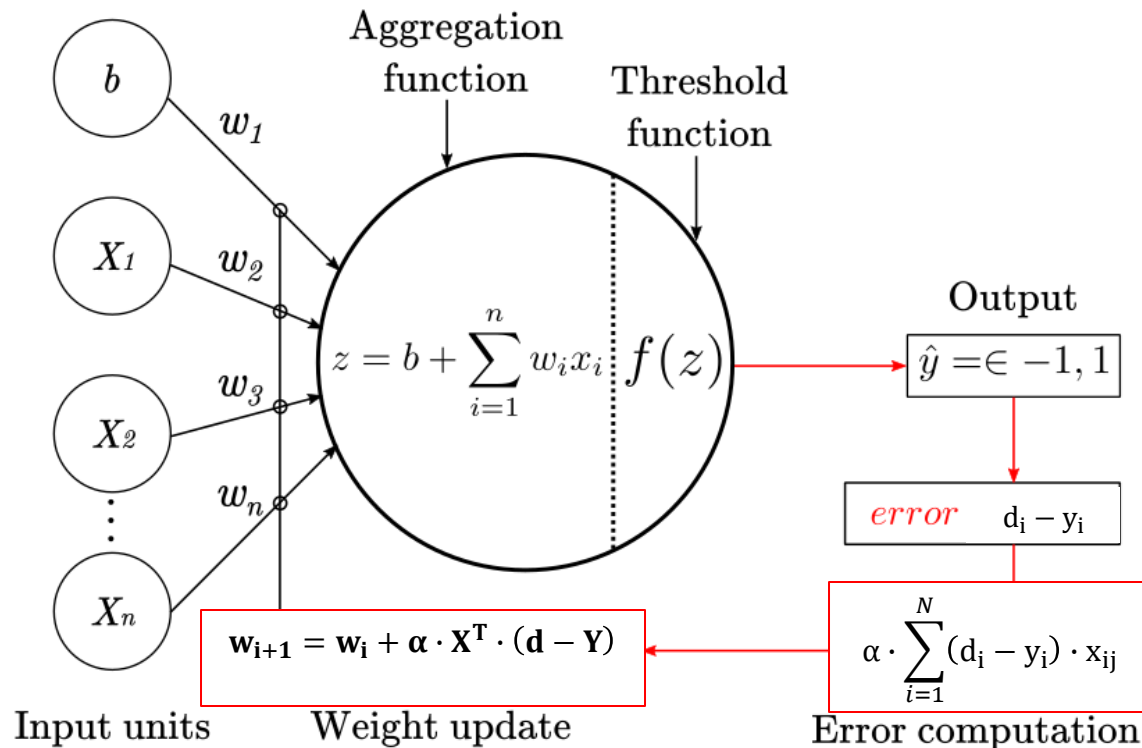
$$\hat{y} = f\left(\sum_{i=1}^N w_i \cdot x_i + w_0\right)$$

Redes Neuronales Perceptrón

El Perceptrón es el modelo más básico de red neuronal artificial, diseñado por Frank Rosenblatt en 1958. Se usa principalmente para clasificación lineal. El perceptrón es un modelo de neurona artificial que toma un conjunto de entradas x , las multiplica por pesos w , y pasa la suma ponderada a través de una función de activación (generalmente el escalón de Heaviside).

$$z = \sum_{j=1}^N w_j \cdot x_i + w_0 \rightarrow \hat{y}_i = F(z) \equiv \begin{cases} 1 & z \geq 0 \\ -1 & z < 0 \end{cases}$$

Perceptron training loop



$$Z = \begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1n} \\ 1 & x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m1} & \cdots & x_{mn} \end{pmatrix} \cdot \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{pmatrix}$$

Se obtiene la función de Heaviside:

$$Y = F(Z)$$

El Perceptrón no usa una función de error diferenciable como el MSE, sino que simplemente actualiza los pesos si hay un error de clasificación. En forma matricial:

$$w_{i+1} = w_i + \alpha \cdot X^T \cdot (d - Y)$$

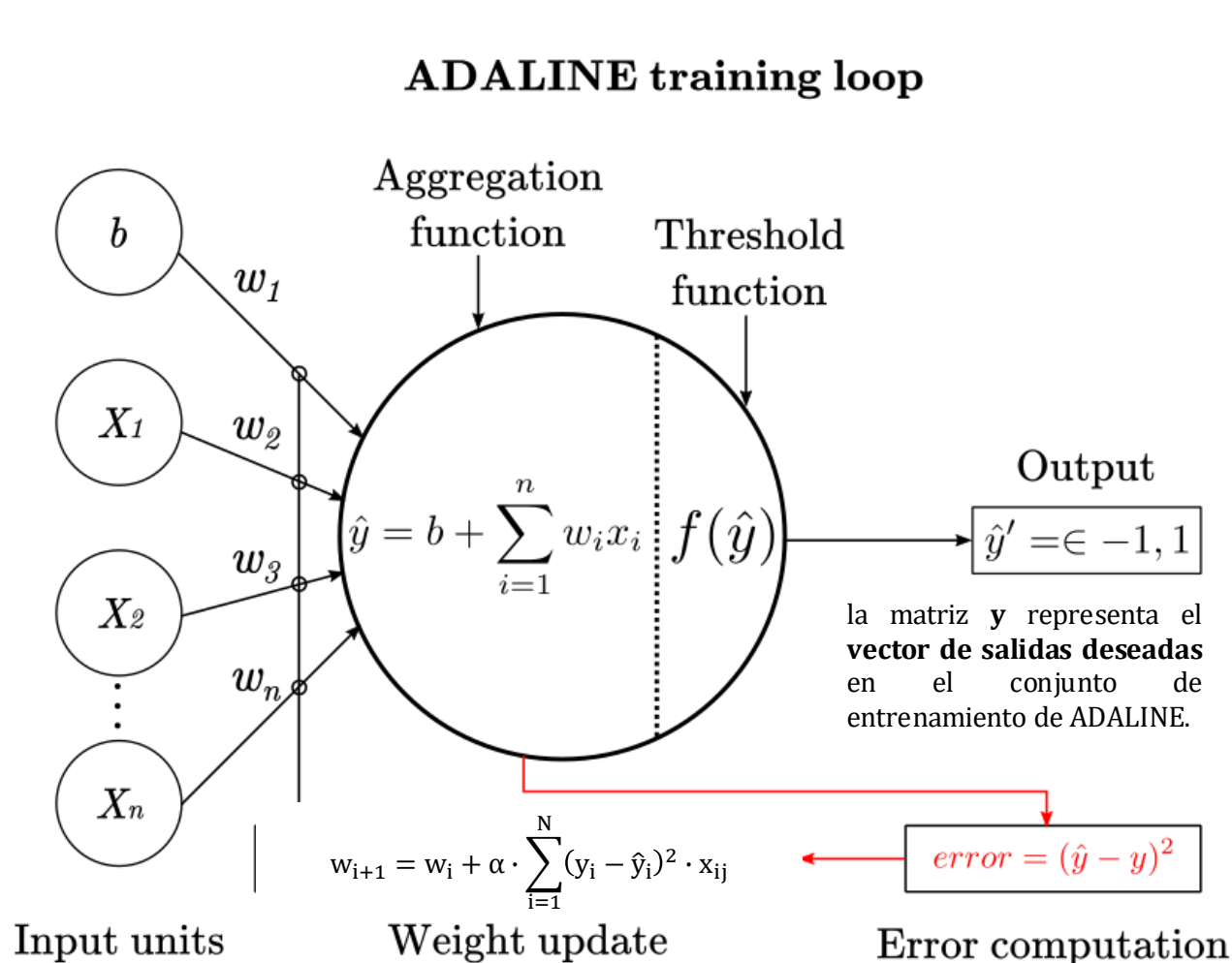
$$w_0 = w_0 + \alpha \cdot \sum_{i=1}^M e_i$$

El perceptrón converge siempre si los datos son linealmente separables. Esto es garantizado por el teorema de convergencia del perceptrón:

$$d \cdot (w^* \cdot x) > 0$$

Donde w es un vector de pesos que clasifica correctamente todas las muestras. Si los datos no son linealmente separables, el perceptrón no converge y oscila indefinidamente.

ADALINE (Adaptive Linear Neuron, o Neurona Adaptativa Lineal) es un modelo de red neuronal supervisada desarrollado por Bernard Widrow y Marcian Hoff en 1960. Es una mejora del Perceptrón y se basa en la minimización del error cuadrático medio (MSE) en lugar de solo corregir clasificaciones erróneas.



$$z = \sum_{j=1}^N w_j \cdot x_{ij} + w_0 \rightarrow \hat{y}_i = F(z) = z$$

Considerando que el error como:

$$\text{Error} = \frac{1}{2} \cdot \sum_{i=1}^N (y_i - \hat{y}_i)^2 \rightarrow \frac{\partial \text{Error}}{\partial w_j} = \sum_{i=1}^N (y_i - \hat{y}_i) \cdot \frac{\partial}{\partial w_j} (-\hat{y}_i)$$

$$\frac{\partial \hat{y}_i}{\partial w_j} = \frac{\partial (\sum_{j=1}^N w_j \cdot x_{ij} + w_0)}{\partial w_j} = x_{ij} \rightarrow \frac{\partial \text{Error}}{\partial w_j} = - \sum_{i=1}^N (y_i - \hat{y}_i) \cdot x_{ij}$$

$$\frac{\partial \text{Error}}{\partial w_0} = - \sum_{i=1}^N (y_i - \hat{y}_i)$$

$$w_j^{(t+1)} = w_j^{(t)} + \alpha \cdot \sum_{i=1}^N (y_i - \hat{y}_i) \cdot x_{ij}$$

$$w_0^{(t+1)} = w_0^{(t)} + \alpha \cdot \sum_{i=1}^N (y_i - \hat{y}_i)$$

Característica	Perceptrón	ADALINE (Adaptive Linear Neuron)
Tipo de salida	Discreta (0 o 1)	Continua antes de activación
Función de activación	Escalón (Heaviside)	Lineal (sin activación antes del aprendizaje)
Regla de aprendizaje	Regla del Perceptrón	Regla de Widrow-Hoff (LMS)
Criterio de ajuste	Basado en el error de clasificación	Basado en el error cuadrático mínimo (MSE)
Convergencia	Solo si los datos son linealmente separables	Converge incluso si los datos no son separables
Corrección de pesos	Solo si la salida discreta es incorrecta	Se ajusta en cada iteración, minimizando el error

Diferencia clave entre Perceptrón y ADALINE

Modelo	Función de Activación	Criterio de Aprendizaje	Optimización
Perceptrón	Escalón de Heaviside	Corrige errores de clasificación	Regla de actualización directa
ADALINE	No usa activación antes del entrenamiento	Minimiza el error cuadrático medio (MSE)	Descenso del gradiente

¿Por qué ADALINE ya no se usa?

1. Es un modelo lineal

1. ADALINE solo puede resolver problemas de clasificación que son **linealmente separables**.
2. No puede manejar datos más complejos que requieran separación no lineal.

2. No aprovecha funciones de activación no lineales

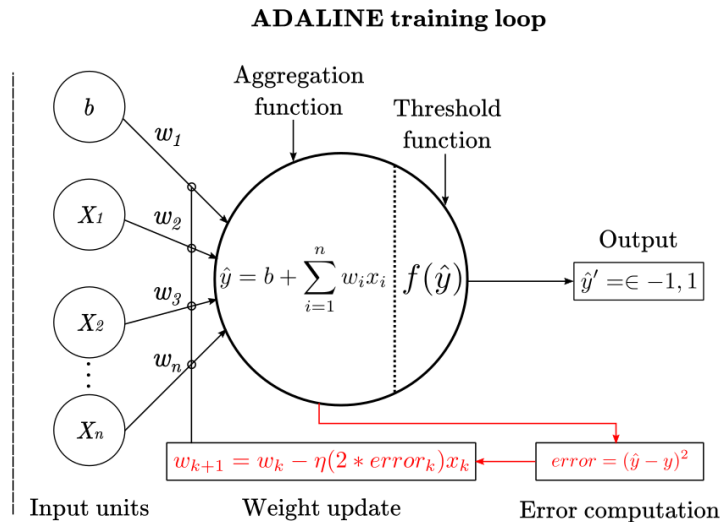
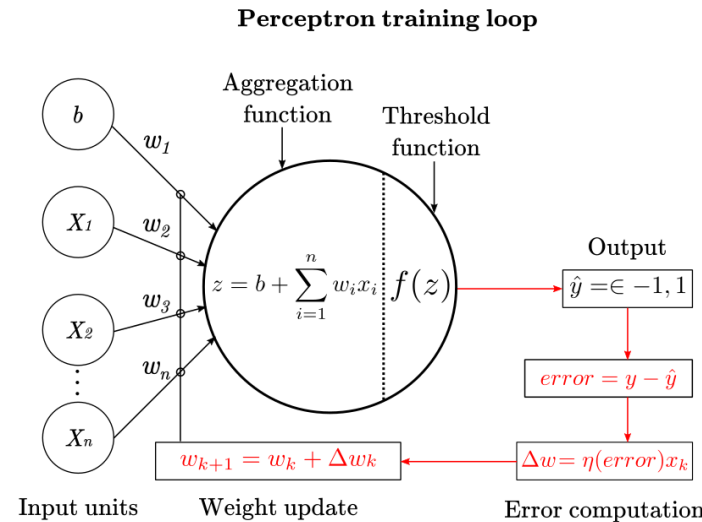
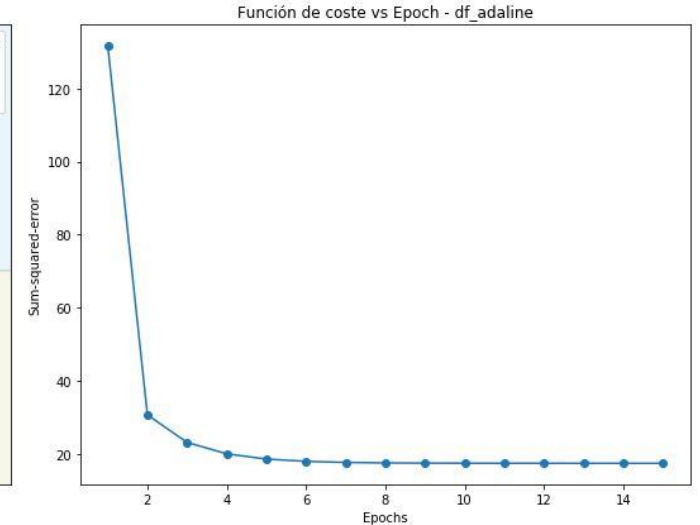
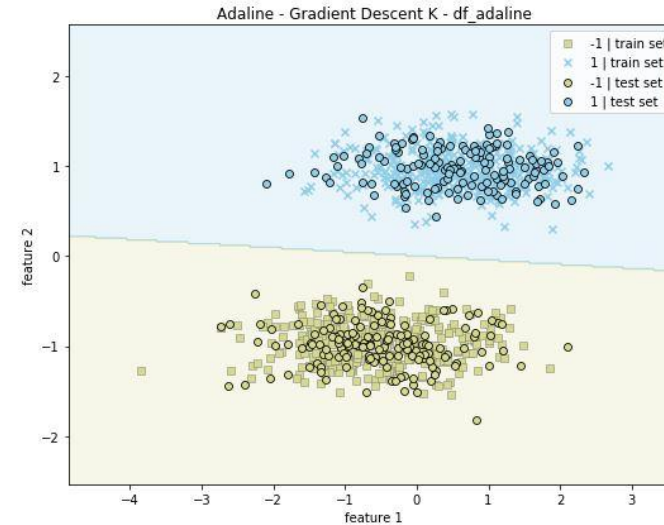
1. Las redes neuronales modernas usan funciones como **ReLU**, **sigmoide** o **tanh**, que permiten representar relaciones más complejas.

3. Las redes neuronales profundas lo han reemplazado

1. Modelos como **Redes Neuronales Artificiales (ANN)**, **Redes Convolucionales (CNN)** y **Transformers** han demostrado ser mucho más poderosos para tareas de clasificación, regresión y reconocimiento de patrones.

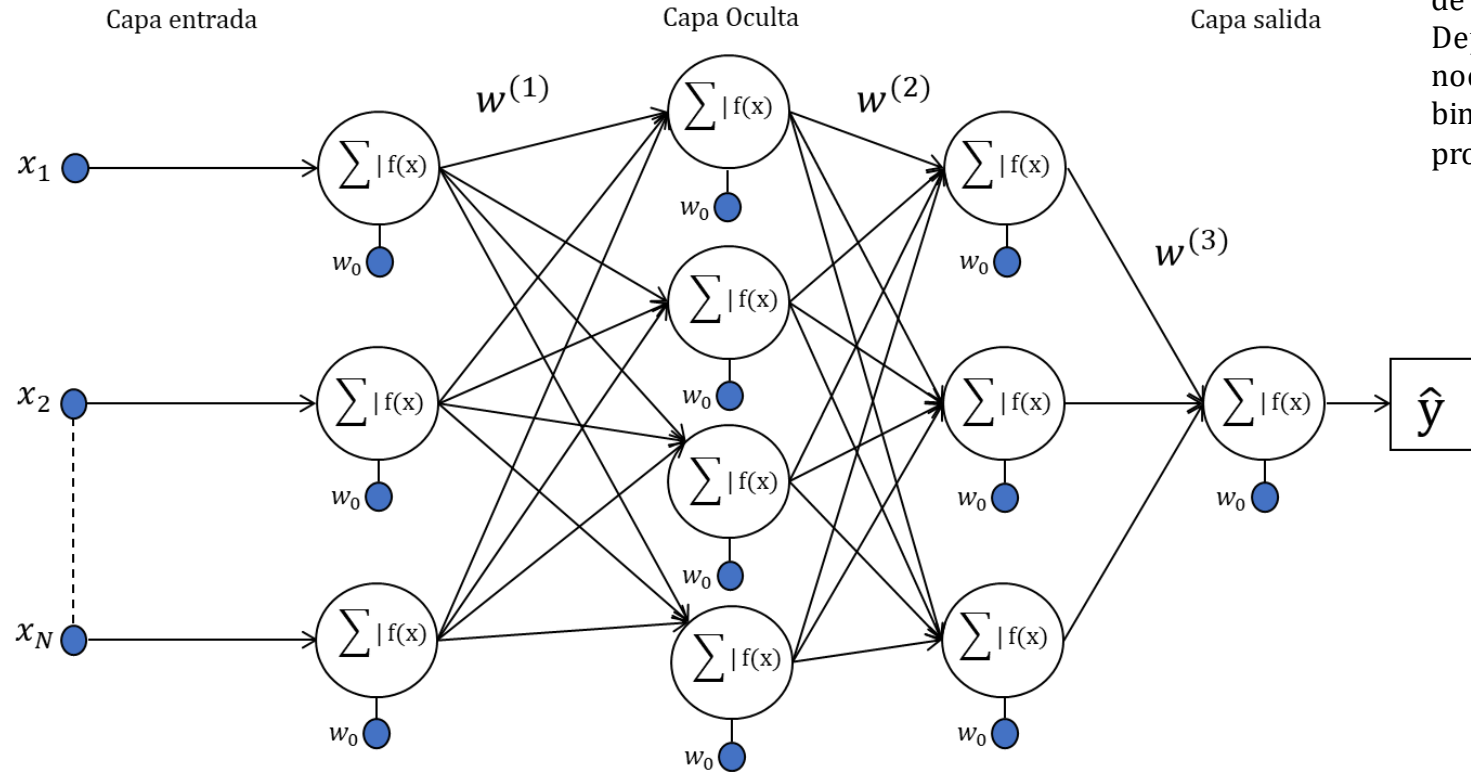
4. Problemas de convergencia con datos grandes

1. ADALINE usa **descenso del gradiente en batch**, lo que puede ser lento para conjuntos de datos grandes. Hoy en día, se prefieren variantes como el **descenso del gradiente estocástico (SGD)**.

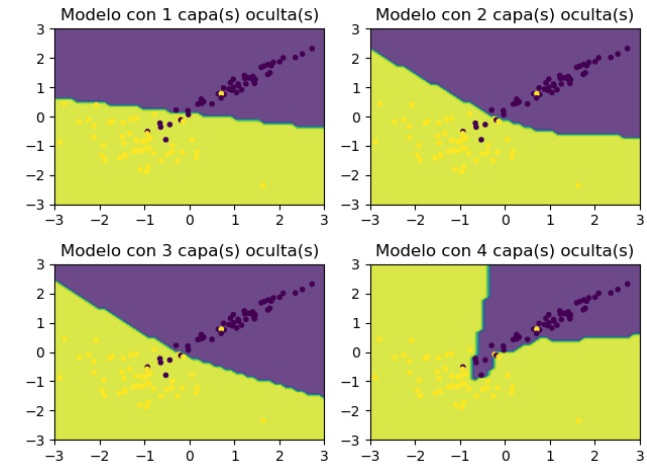


Redes neuronales multicapa

Capa de entrada (Input Layer): Esta es la primera capa de la red, que recibe los datos de entrada. Cada nodo en esta capa representa una característica del conjunto de datos.



Capa de salida (Output Layer): Esta es la última capa de la red y produce el resultado final del modelo. Dependiendo del problema, puede tener uno o más nodos. Por ejemplo, para una tarea de clasificación binaria, podría haber un solo nodo que representa la probabilidad de una clase.



Capas ocultas (Hidden Layers): Estas son las capas intermedias entre la capa de entrada y la capa de salida. Una red neuronal multinodal puede tener una o más capas ocultas, lo que la hace más "profunda" (de ahí el término "deep learning"). Cada nodo en una capa oculta toma una combinación ponderada de los valores de los nodos de la capa anterior, los pasa por una función de activación no lineal, y envía el resultado a la siguiente capa.


```
▶ my_first_neural_net = tf.keras.Sequential([
    layers.Input(shape=(3, ), name="input"),
    layers.Dense(units=5, activation="sigmoid", name="hidden"),
    layers.Dense(units=2, activation="softmax", name="output")
])
```

```
▶ my_first_neural_net.summary()
```

☞ Model: "sequential"

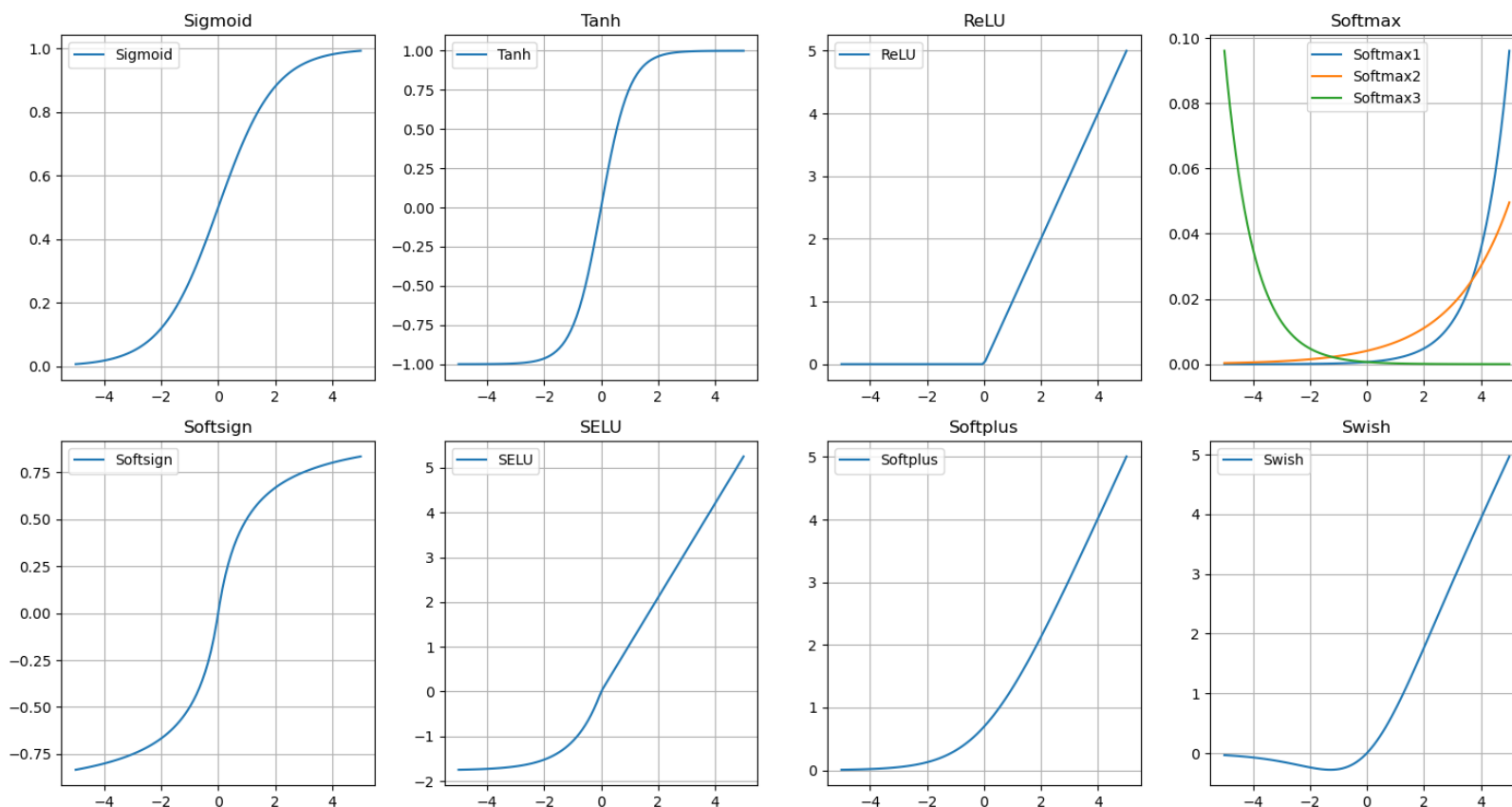
Layer (type)	Output Shape	Param #
=====		
hidden (Dense)	(None, 5)	20
output (Dense)	(None, 2)	12
=====		
Total params: 32		
Trainable params: 32		
Non-trainable params: 0		

Función de activación

¿Qué es una función de activación? $\hat{y} = f\left(\sum_{i=1}^N w_i \cdot x_i + w_0\right)$

Una **función de activación** es una transformación matemática aplicada a la salida de una neurona en una red neuronal. Su propósito es **introducir no linealidad** en la red para que pueda aprender relaciones complejas en los datos. Si no se usara una función de activación, la red neuronal se comportaría como una simple regresión lineal.

Nombre	Fórmula
Sigmoidea	$f(x) = \frac{1}{1 + e^{-x}}$
Tanh	$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
Relu	$f(x) = \max\{0, x\}$
Softmax	$f(x) = \frac{e^x}{\sum_{i=1}^N e^{x_i}}$
Softsign	$f(x) = \frac{1}{1 + x }$
Selu	$f(x) = \begin{cases} \alpha \cdot (e^x - 1), & x < 0 \\ x, & x \geq 0 \end{cases}$
Softplus	$f(x) = \log(1 + e^x)$
Swish	$f(x) = \frac{x}{1 + e^{-x}}$



Optimización y función de pérdida

Optimizadores

Descenso del gradiente estocástico (Stochastic Gradient Descent, SGD)

$$\theta^t = \theta^{t-1} - \alpha \cdot \nabla J_i(\theta)$$

Momentum

$$v = \beta \cdot v + (1 - \beta) \cdot \nabla J(\theta)$$

Adagrad

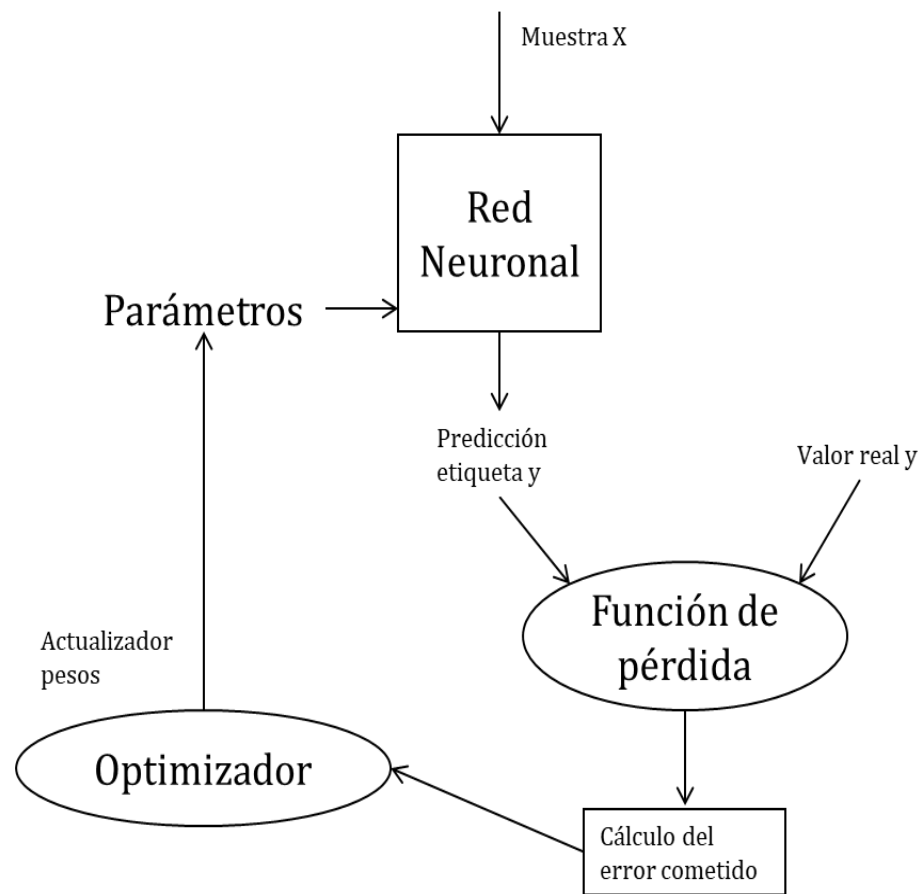
$$\theta^t = \theta^{t-1} - \frac{\alpha}{\sqrt{F_i + \epsilon}} \cdot \nabla J(\theta)$$

Adam

$$\begin{cases} \theta^t = \theta^{t-1} - \frac{\alpha}{\sqrt{v + \epsilon}} \cdot m \\ m = \beta_1 \cdot m + (1 - \beta_1) \cdot \nabla J(\theta) \\ v = \beta_2 \cdot v + (1 - \beta_2) \cdot (\nabla J(\theta))^2 \end{cases}$$

RMSprop

$$\begin{cases} \theta^t = \theta^{t-1} - \frac{\alpha}{\sqrt{E(\nabla J(\theta)^2) + \epsilon}} \cdot \nabla J(\theta) \\ E(\nabla J(\theta)^2)_t = \beta_1 \cdot E(\nabla J(\theta)^2)_{t-1} + (1 - \beta_1) \cdot E(\nabla J(\theta)^2)_{t-1} \end{cases}$$



Funciones de pérdida

El error absoluto medio (MAE)

$$MSE = \frac{1}{N} \cdot \sum_{i=1}^N (y_i - \hat{y}_i)$$

La entropía cruzada

$$\begin{cases} \text{Binaria: } CE = -\frac{1}{N} \cdot \sum_{i=1}^N [y_i \cdot \log|\hat{y}_i| + (1 - y_i) \cdot \log|1 - \hat{y}_i|] \\ \text{Multiclase: } CE = -\frac{1}{N} \cdot \sum_{i=1}^N \sum_{j=1}^M y_{i,j} \cdot \log|\hat{y}_{i,j}| \end{cases}$$

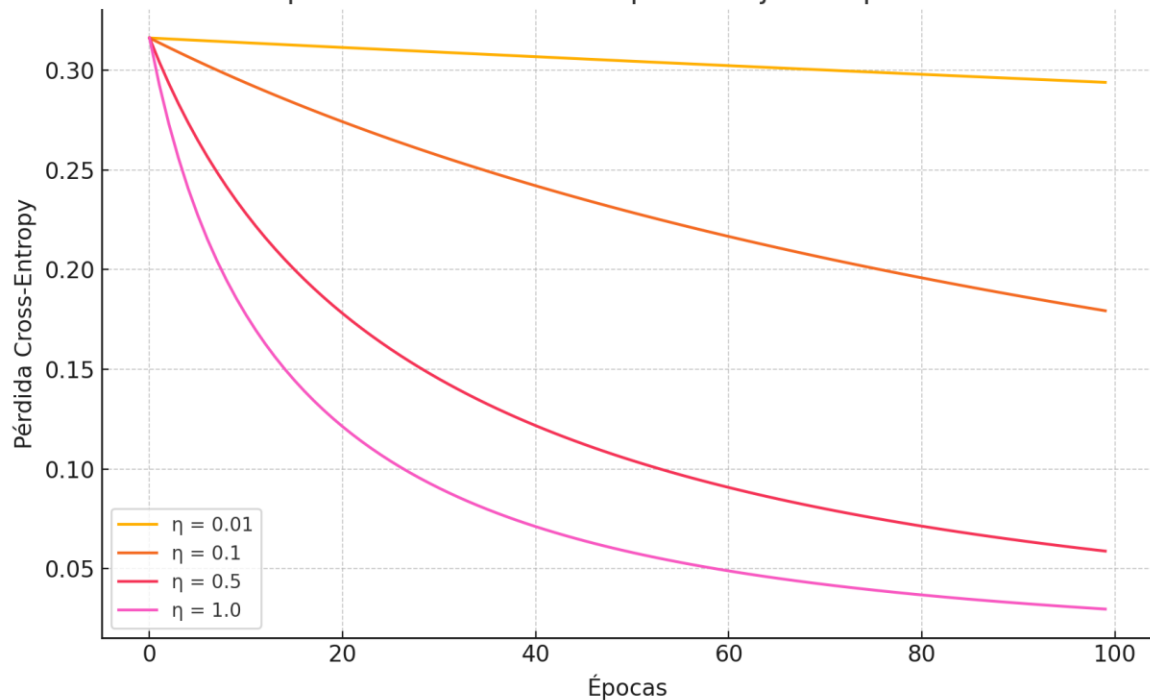
La pérdida de Huber

$$H = -\frac{1}{N} \cdot \sum_{i=1}^N \begin{cases} (y_i - \hat{y}_i)^2 & \text{para } |y_i - \hat{y}_i| \leq \delta \\ \delta \cdot (|y_i - \hat{y}_i|) - \frac{1}{2} \cdot \delta^2 & \text{para } |y_i - \hat{y}_i| > \delta \end{cases}$$

La **tasa de aprendizaje** (η o α) es un hiperparámetro que controla el tamaño del paso que da el algoritmo de optimización en cada iteración para actualizar los pesos de la red neuronal. **Un valor muy grande** puede hacer que el modelo no converja o salte sobre el mínimo óptimo. **Un valor muy pequeño** puede hacer que el entrenamiento sea muy lento y se estanque en un mínimo local.

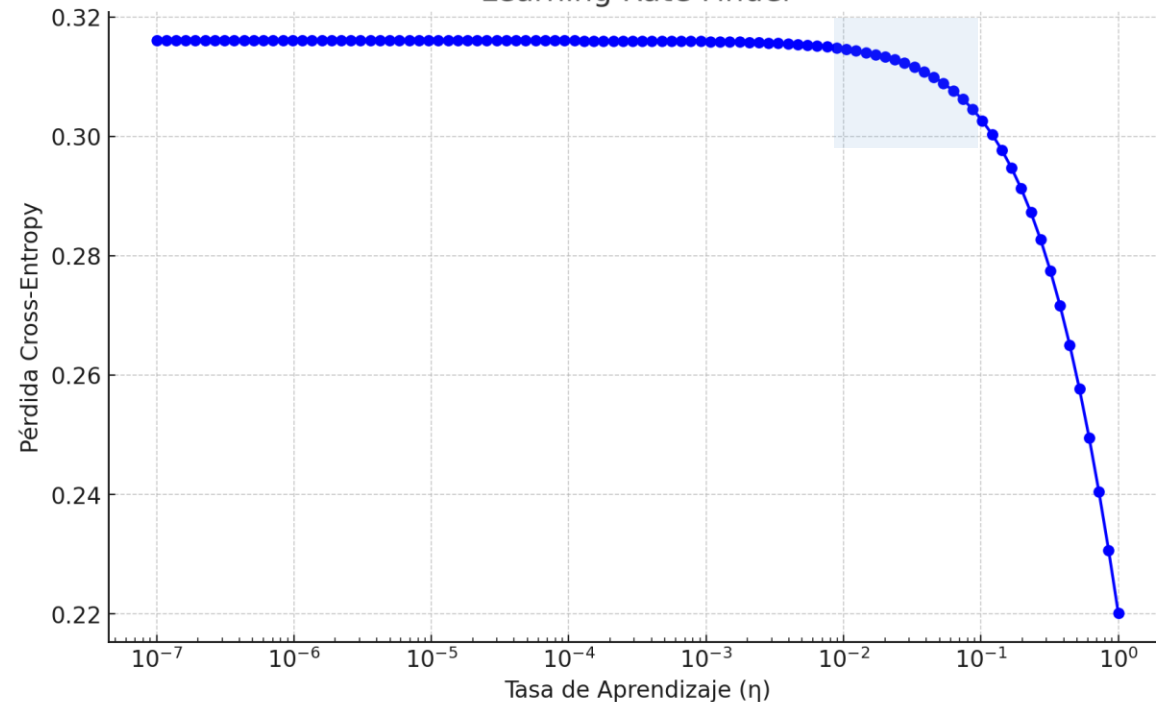
Método	Tipo de Optimización	Ajuste de η	Valores Típicos de η	Cuándo Usarlo
SGD (Stochastic Gradient Descent)	Basado en gradiente	Fijo	10E-02 a 10E-03	Modelos pequeños, datos bien normalizados, entrenamientos rápidos
Momentum SGD	Gradiente con memoria	Fijo, con acumulación	10E-02 a 10E-03 ($\beta=0.9$)	Cuando los gradientes oscilan demasiado, útil en redes profundas
AdaGrad (Adaptive Gradient Algorithm)	Adaptativo	Disminuye con el tiempo	10E-02 a 10E-03	Para datos dispersos o NLP (donde algunas features son raras)
RMSprop (Root Mean Square Propagation)	Adaptativo	Ajusta η dinámicamente	10E-03 a 10E-04 ($\beta=0.9$)	Redes recurrentes (RNN, LSTM) y entrenamientos largos
Adam (Adaptive Moment Estimation)	Adaptativo + Momento	Ajusta η dinámicamente	10E-03 a 10E-04 ($\beta=0.9$)	Más usado en deep learning, CNN, NLP, datos generales
AdamW (Adam con Decaimiento de Pesos)	Adam mejorado	Ajusta η dinámicamente	10E-03 a 10E-04	Similar a Adam, pero mejor para regularización en redes profundas

Comparación de tasas de aprendizaje en optimización



Un η muy bajo (0.01) hace que el entrenamiento sea lento. Un η intermedio (0.1 - 0.5) proporciona un buen balance entre estabilidad y velocidad. Un η demasiado alto podría hacer que la pérdida oscile o no converja.

Learning Rate Finder

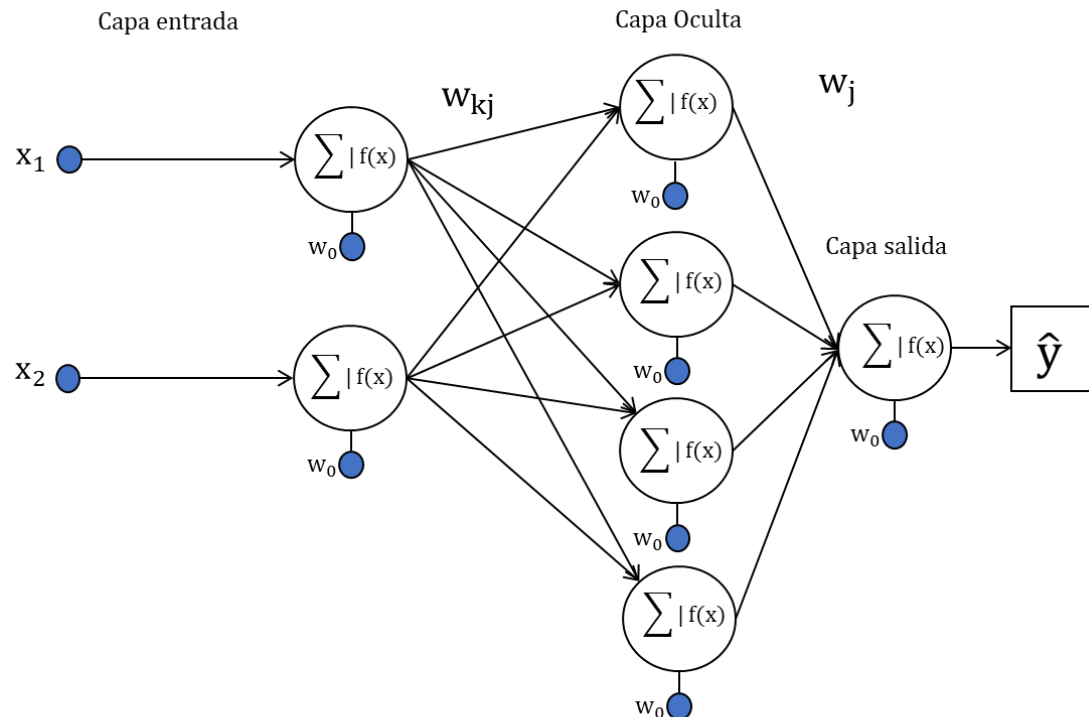


Para elegir la mejor tasa de aprendizaje, se selecciona el valor en el que la pérdida comienza a disminuir rápidamente antes de volverse inestable.

Forward Propagation

El **Forward Propagation** es el proceso mediante el cual una red neuronal calcula la salida a partir de las entradas y los pesos en cada capa. A continuación, te explico **paso a paso** las matemáticas detrás del **propagado hacia adelante (Forward Propagation)** en una red con **una capa oculta**.

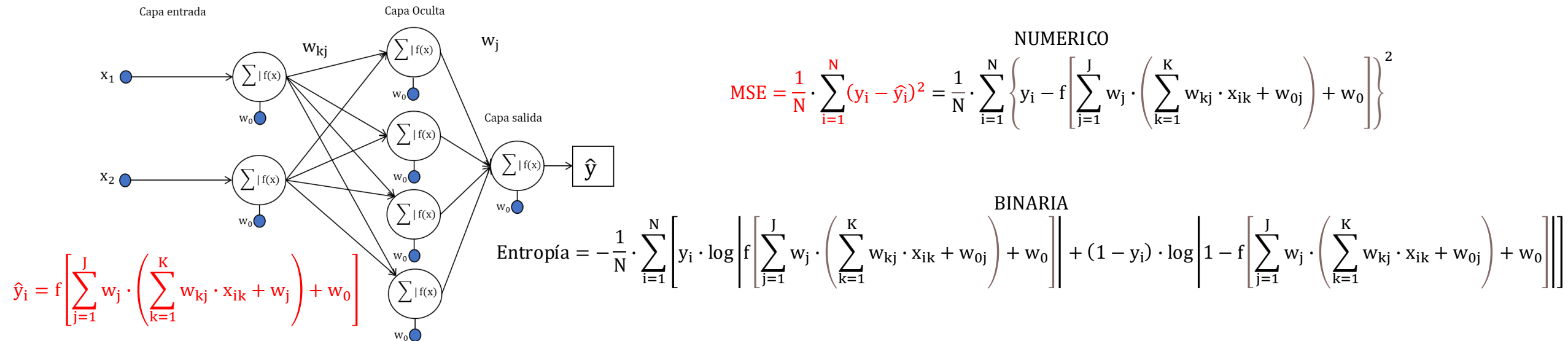
$$\hat{y}_i = f \left[\sum_{j=1}^J w_j \cdot \left(\sum_{k=1}^K w_{kj} \cdot x_{ik} + w_j \right) + w_0 \right] \rightarrow \text{Oculta}(a) = f \left(\sum_{k=1}^K w_{kj} \cdot x_k + w_{0j} \right) \rightarrow z = \sum_{j=1}^J w_j \cdot a_j + w_0 \rightarrow \hat{y}_i = f(z)$$



Un **epoch** es una **iteración completa a través del conjunto de datos de entrenamiento** en una red neuronal. Es decir, durante un **epoch**, el modelo procesa **todas las muestras** de entrenamiento **una vez** y ajusta sus pesos mediante **backpropagation** y optimización.

Backpropagation

La **backpropagation** es el proceso de ajustar los pesos en una red neuronal mediante la propagación del error desde la salida hacia atrás a través de las capas. Calcula los gradientes del error con respecto a cada peso usando la **regla de la cadena** y luego actualiza los pesos para minimizar el error. Este proceso se repite en múltiples iteraciones para que la red aprenda a hacer predicciones más precisas.



El algoritmo consiste en ir iterando los pesos en cada una de las etapas con un parámetro α o tasa de aprendizaje (en el caso del descenso del gradiente):

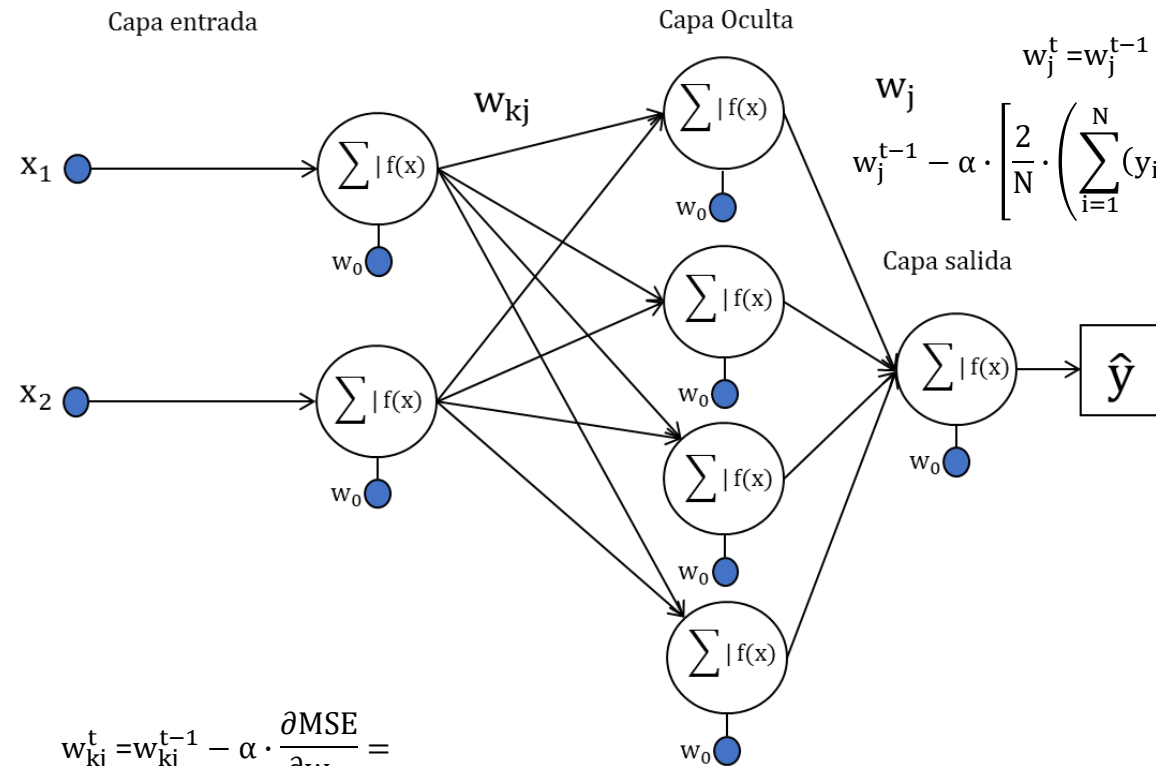
$$\begin{cases} w_j^t = w_j^{(t-1)} - \alpha \cdot \frac{\partial \text{Error}}{\partial w_j} \\ w_{kj}^t = w_{kj}^{(t-1)} - \alpha \cdot \frac{\partial \text{Error}}{\partial w_{kj}} \\ w_0^t = w_0^{(t-1)} - \alpha \cdot \frac{\partial \text{Error}}{\partial w_0} \end{cases}$$

Backpropagation

$$\frac{\partial \text{MSE}}{\partial w_{kj}} = -\frac{2}{N} \cdot \left(\sum_{i=1}^N (y_i - \hat{y}_i)^1 \right) \cdot \left(x_{ik} \cdot \sum_{k=1}^K w_j \right)$$

$$\frac{\partial \text{MSE}}{\partial w_j} = -\frac{2}{N} \cdot \left(\sum_{i=1}^N (y_i - \hat{y}_i)^1 \right) \cdot \left(\sum_{k=1}^K w_{kj} \cdot x_{ik} \right)$$

Demasiadas formulas?
Os facilito la vida 😊



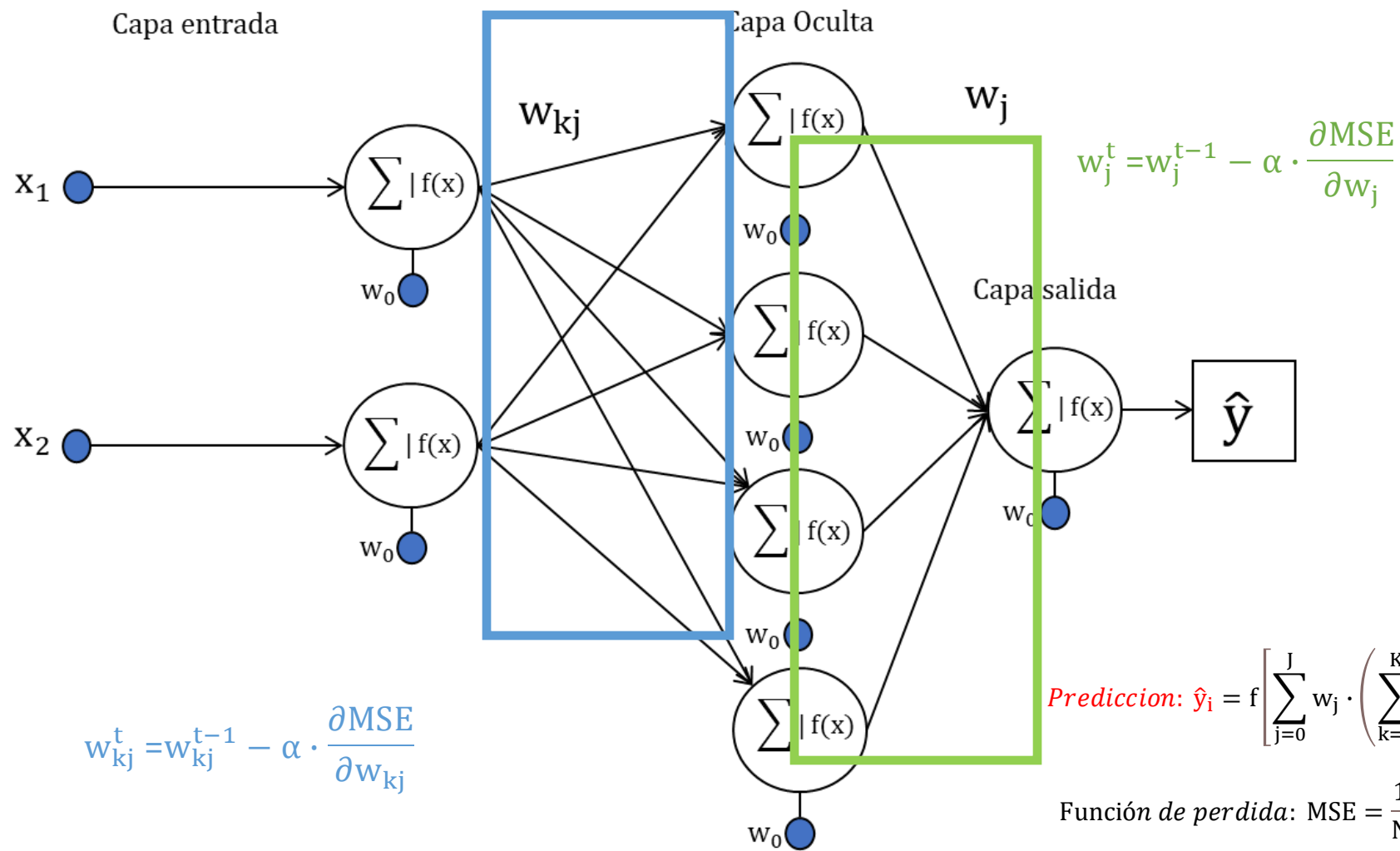
$$w_j^t = w_j^{t-1} - \alpha \cdot \frac{\partial \text{MSE}}{\partial w_j} = w_j^{t-1} - \alpha \cdot \left[\frac{2}{N} \cdot \left(\sum_{i=1}^N (y_i - \hat{y}_i)^1 \right) \cdot \left(\sum_{k=1}^K w_{kj} \cdot x_{ik} \right) \right]$$

$$\hat{y}_i = f \left[\sum_{j=0}^J w_j \cdot \left(\sum_{k=0}^K w_{kj} \cdot x_{ik} \right) \right] \approx \sum_{j=0}^J w_j \cdot \left(\sum_{k=0}^K w_{kj} \cdot x_{ik} \right)$$

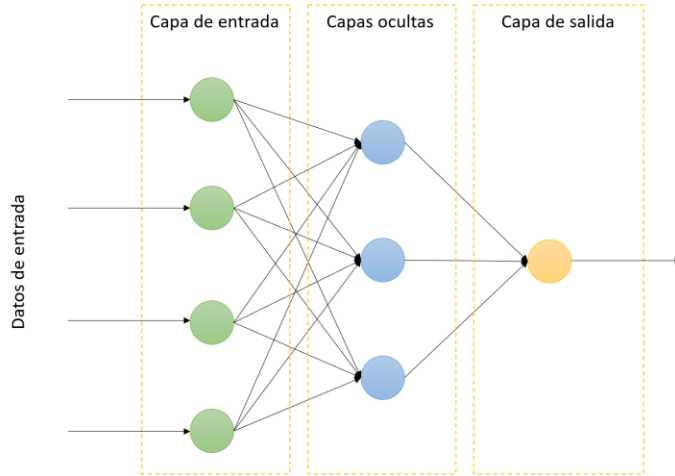
$$w_{kj}^t = w_{kj}^{t-1} - \alpha \cdot \frac{\partial \text{MSE}}{\partial w_{kj}} = w_{kj}^{t-1} - \alpha \cdot \left[\frac{2}{N} \cdot \left(\sum_{i=1}^N (y_i - \hat{y}_i)^1 \right) \cdot \left(x_{ik} \cdot \sum_{k=1}^K w_j^{t-1} \right) \right]$$

$$\text{MSE} = \frac{1}{N} \cdot \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \frac{1}{N} \cdot \sum_{i=0}^N \left\{ y_i - \sum_{j=0}^J w_j \cdot \left(\sum_{k=0}^K w_{kj} \cdot x_{ik} \right) \right\}^2$$

Backpropagation



Red Neuronal con Múltiples Entradas y una salida

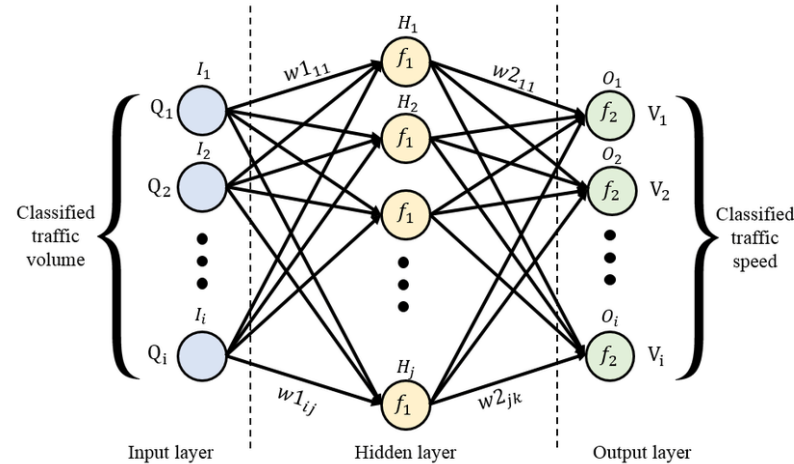


Esta arquitectura recibe varias entradas y produce una única salida. Es común en problemas de regresión o clasificación donde múltiples características determinan un solo resultado.

Se usa en:

1. Modelos de regresión (predicción de precios de casas, temperatura, ventas).
2. Clasificación binaria (detección de fraude, diagnóstico médico con múltiples variables).

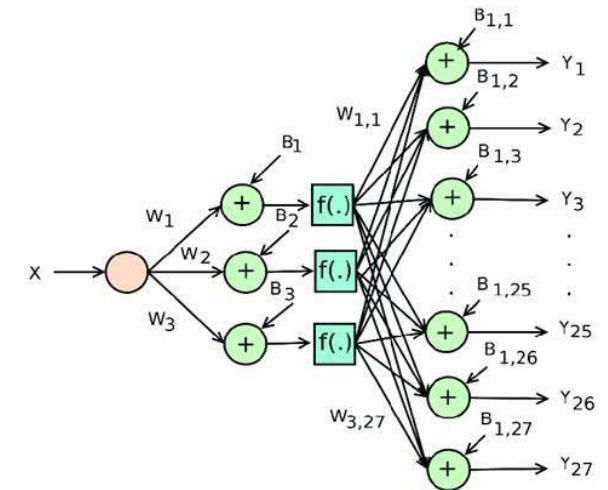
Red Neuronal con Múltiples Entradas y Múltiples Salidas



Esta arquitectura maneja múltiples entradas y genera múltiples salidas, adecuada para sistemas complejos que requieren procesar diversas fuentes de información y producir diferentes tipos de resultados.

1. Modelos en salud (entradas: datos médicos, imágenes → salidas: diagnóstico, pronóstico, tratamiento recomendado).
2. Sistemas financieros (datos económicos, comportamiento del usuario → predicción de riesgo y recomendación de inversión).
3. Redes neuronales en robótica y automatización (sensores múltiples → decisiones de control).

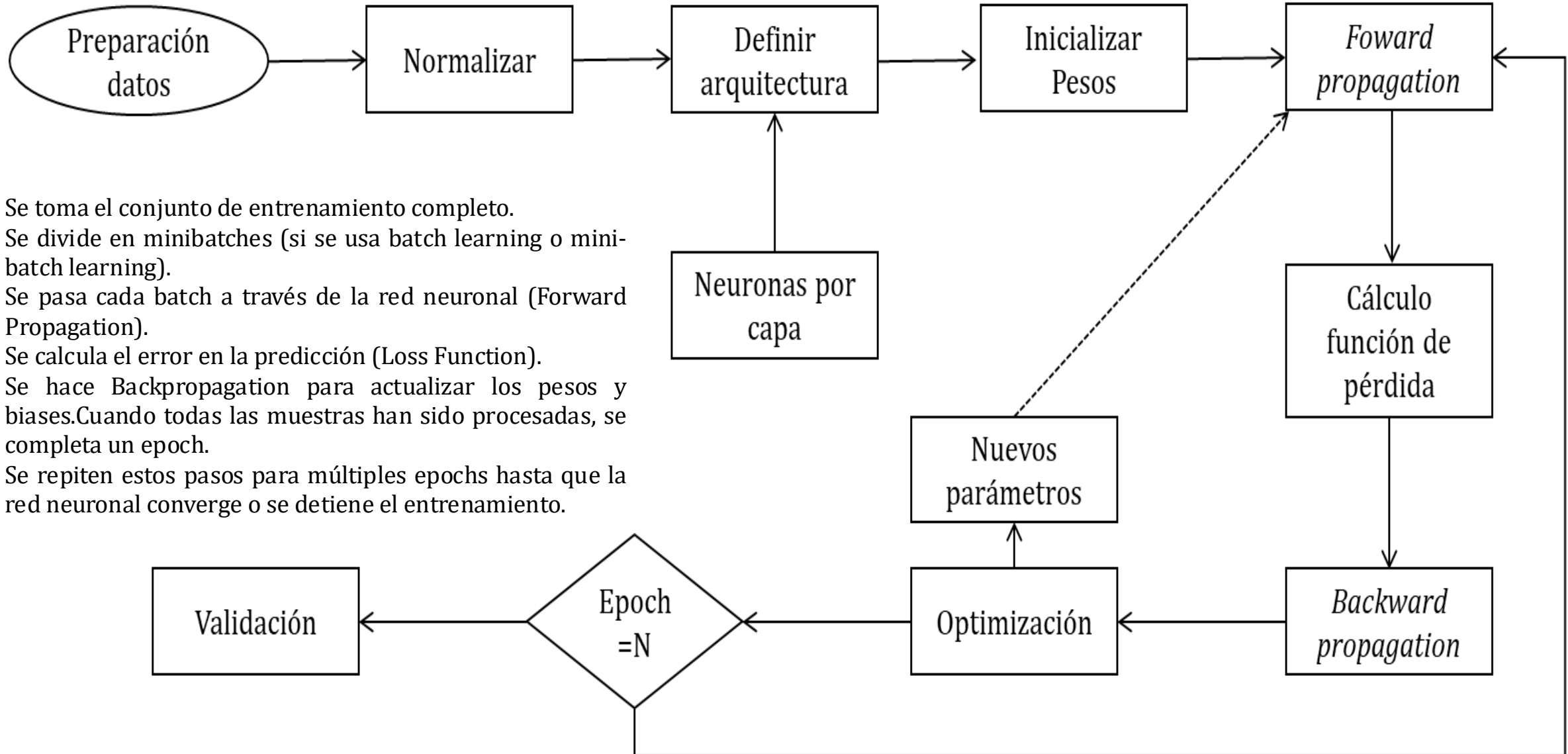
Red Neuronal con una entrada y múltiples salidas



En esta configuración, una sola entrada se procesa para generar múltiples salidas. Es útil en tareas donde una entrada debe ser evaluada en varios aspectos o categorías.

1. Sistemas de recomendación (dado un usuario, predecir varias preferencias).
2. Modelos de predicción multitarea (un texto → predicción de categoría + tono emocional + relevancia).
3. Generación de múltiples atributos de una imagen o señal.

Proceso de construcción de una red neuronal



1. Se toma el conjunto de entrenamiento completo.
2. Se divide en minibatches (si se usa batch learning o mini-batch learning).
3. Se pasa cada batch a través de la red neuronal (Forward Propagation).
4. Se calcula el error en la predicción (Loss Function).
5. Se hace Backpropagation para actualizar los pesos y biases. Cuando todas las muestras han sido procesadas, se completa un epoch.
6. Se repiten estos pasos para múltiples epochs hasta que la red neuronal converge o se detiene el entrenamiento.

Código Python de una red multicapa

Definir la arquitectura de la red neuronal

```
# Definir el modelo
model = Sequential()
model.add(Dense(16, activation='relu',
input_dim=X_train.shape[1]))
model.add(Dense(1, activation='sigmoid'))

# Compilar el modelo
model.compile(optimizer=Adam(learning_rate=0.001),
loss='binary_crossentropy', metrics=['accuracy'])
model.summary()

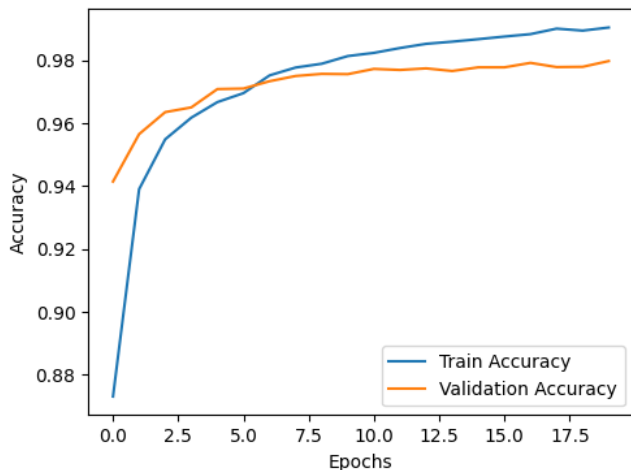
# Entrenar el modelo
history = model.fit(X_train, y_train, epochs=50,
batch_size=32, validation_split=0.2)
```

Compilación del modelo y definición del modelo de compilación

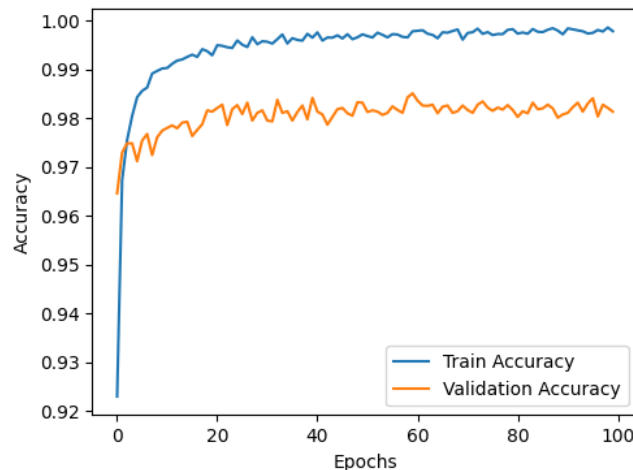
Entrenamiento del modelo

Evaluación de redes neuronales

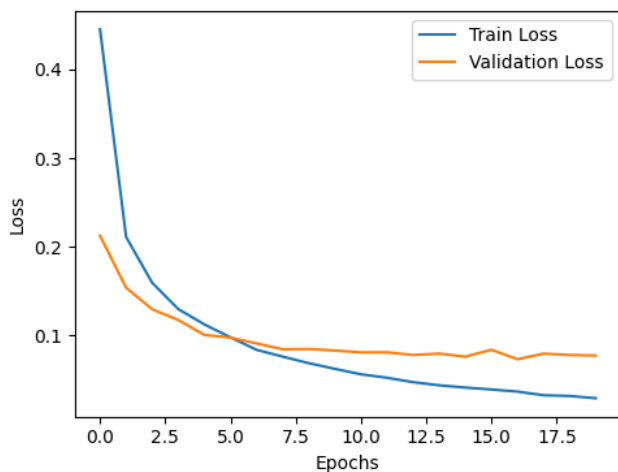
Bien Entrenado



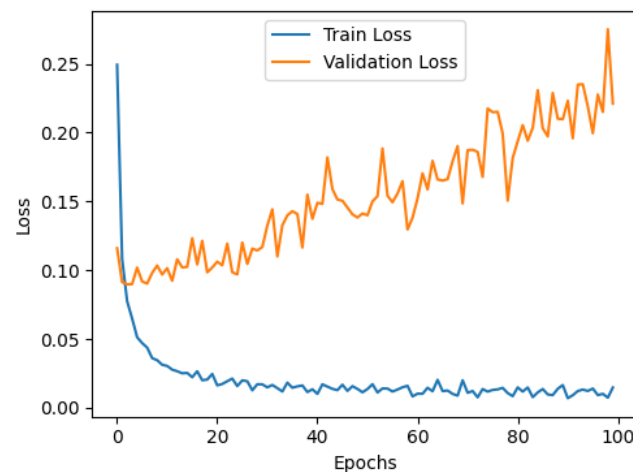
Sobreentrenado



Bien Entrenado



Sobreentrenado



Regularización L1 (Lasso)

Añade una penalización al valor absoluto de los coeficientes de los pesos del modelo. Esta técnica promueve la esparsidad, lo que significa que puede llevar a que algunos coeficientes se reduzcan a cero, eliminando así algunas características del modelo.

Regularización L2 (Ridge)

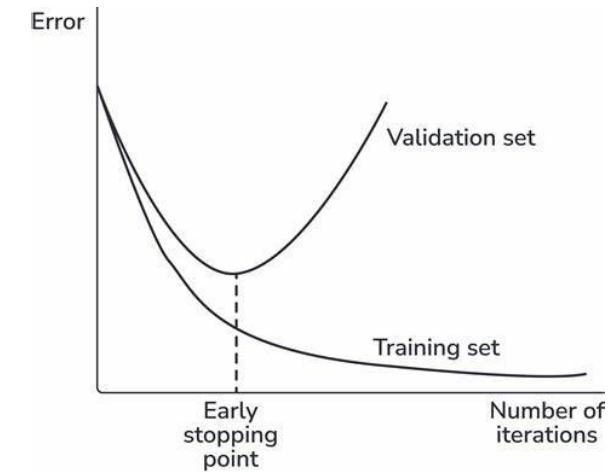
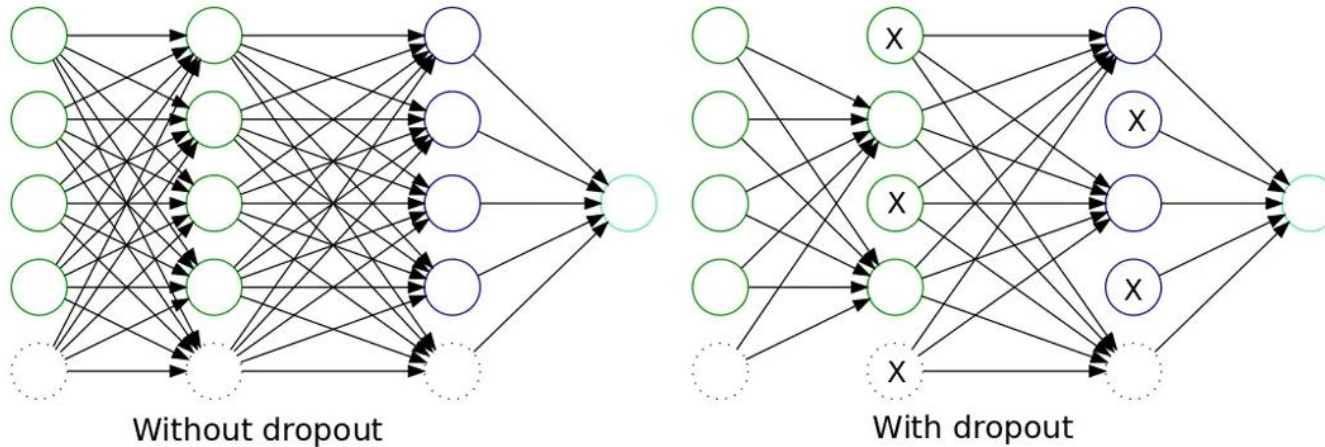
Añade una penalización al cuadrado de los coeficientes de los pesos del modelo. Esto conduce a soluciones con coeficientes pequeños pero no necesariamente a cero, lo que reduce la complejidad del modelo.

Dropout

Durante el entrenamiento, en cada iteración, se desactivan (se establecen en cero) aleatoriamente un porcentaje de neuronas en la red. Esto fuerza a la red a no depender demasiado de ninguna neurona en particular y promueve la creación de representaciones más robustas.

Early Stopping

Durante el entrenamiento, se monitoriza el rendimiento del modelo en un conjunto de validación. Si el rendimiento deja de mejorar durante un cierto número de épocas consecutivas, el entrenamiento se detiene. Esto evita que el modelo se entrene demasiado y se sobreajuste a los datos de entrenamiento.

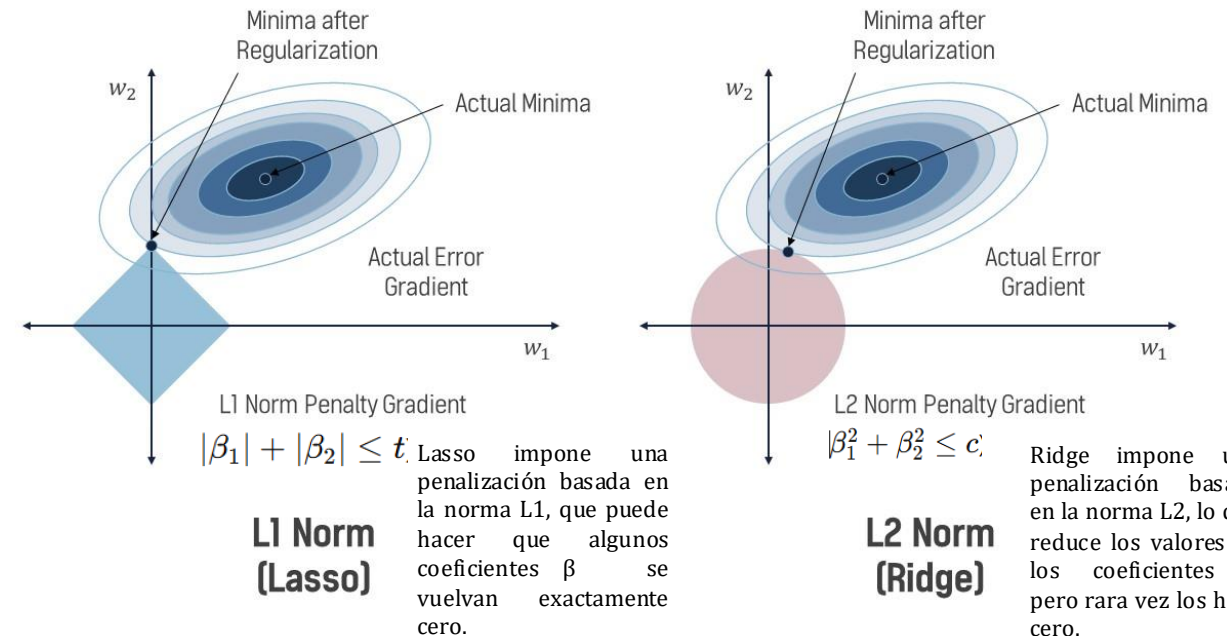


Representan las curvas de nivel de la función de costo de regresión lineal.

La intersección de estas curvas con las restricciones (rombo o círculo) determina los valores óptimos de β .

En Lasso, la intersección tiende a ocurrir en los vértices (lo que lleva a coeficientes cero).

En Ridge, la intersección ocurre dentro del círculo, lo que reduce los coeficientes pero no los anula.

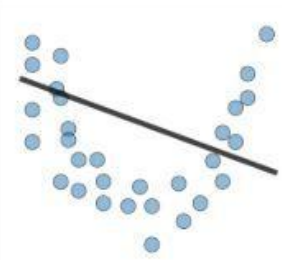
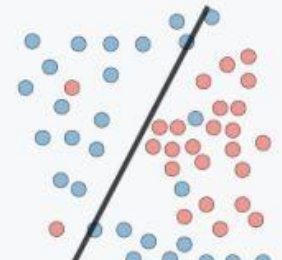
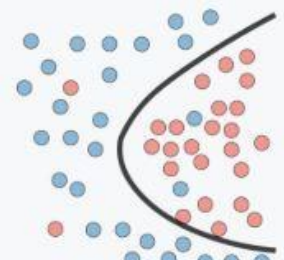
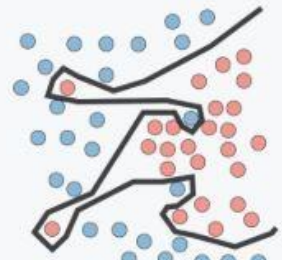
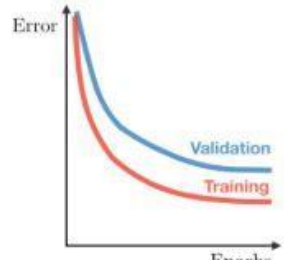


Hiperparámetros

Hiperparámetros de la Topología de la Red Neuronal	Hiperparámetros del Proceso de Aprendizaje
Número de Capas (Depth): Más capas permiten capturar características complejas, pero aumentan el riesgo de sobreajuste y los requisitos computacionales.	Tasa de Aprendizaje (Learning Rate): Afecta la velocidad y estabilidad de la convergencia; debe balancearse entre rapidez y precisión
Número de Neuronas por Capa (Width): Más neuronas ayudan a capturar detalles complejos, pero también incrementan el riesgo de sobreajuste y los costos computacionales.	Tamaño del Lote (Batch Size): Influye en la estabilidad y eficiencia del entrenamiento; lotes grandes mejoran la estabilidad, lotes pequeños mejoran la generalización
Funciones de Activación: Determinan cómo se combinan y transmiten las señales a través de la red, afectando el rendimiento y la convergencia del modelo.	Número de Épocas (Epochs): Más épocas permiten aprender mejor, pero aumentan el riesgo de sobreajuste
Arquitectura Específica: La estructura de la red debe adaptarse al tipo de problema (CNN para imágenes, RNN para secuencias).	Algoritmo de Optimización: La elección del optimizador (SGD, Adam, etc.) impacta la velocidad y calidad de la convergencia
	Regularización: Técnicas como L2, dropout y batch normalization ayudan a prevenir el sobreajuste y mejorar la generalización
	Momento (Momentum): Acelera la convergencia y mejora la estabilidad en problemas con gradientes variables

Balanceo y problemas de ajuste

—

	Underfitting	Just right	Overfitting
Symptoms	- High training error - Training error close to test error - High bias	Training error slightly lower than test error	- Very low training error - Training error much lower than test error - High variance
Regression illustration			
Classification illustration			
Deep learning illustration			
Possible remedies	- Complexify model - Add more features - Train longer		- Perform regularization - Get more data

El **sobreajuste (overfitting)** ocurre cuando un modelo aprende demasiado bien los detalles y peculiaridades (incluido el ruido) del conjunto de datos de entrenamiento, a tal punto que pierde su capacidad de generalizar a datos nuevos.

Características:

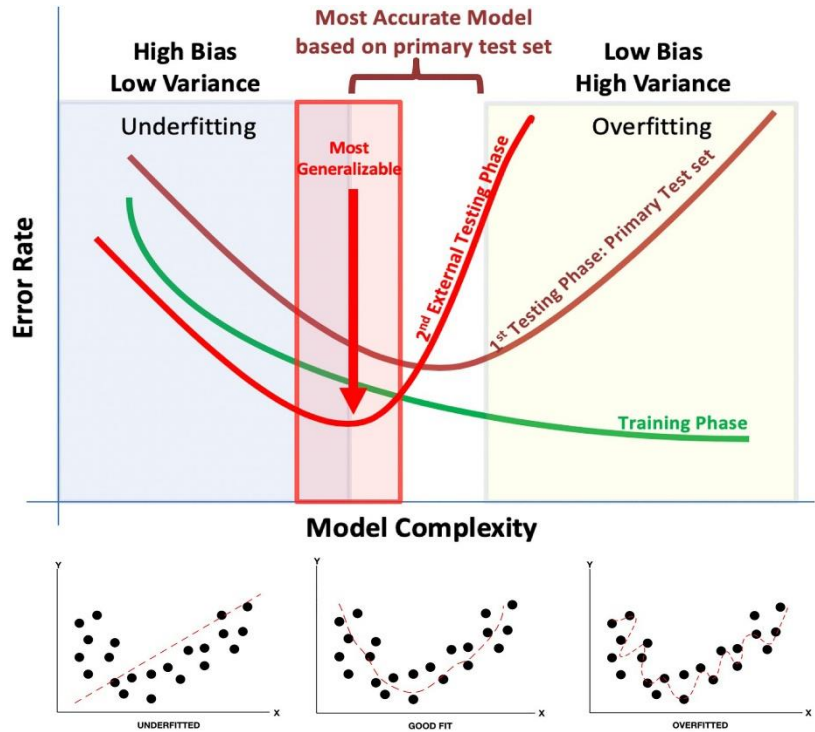
- Desempeño excelente en los datos de entrenamiento.
- Desempeño deficiente en los datos de prueba o validación.
- El modelo se adapta incluso a variaciones irrelevantes o ruido en los datos de entrenamiento.

El **subajuste (underfitting)** ocurre cuando un modelo es demasiado simple para capturar los patrones subyacentes en los datos. Esto significa que no logra ajustarse adecuadamente ni a los datos de entrenamiento ni a los de prueba, resultando en un desempeño deficiente en ambos casos.

Características:

- El modelo no puede capturar las relaciones clave en los datos, lo que genera un error alto en ambos conjuntos.
- Modelo demasiado simple
- Alta generalización, pero baja precisión

El **biasvariance tradeoff** es un concepto fundamental en aprendizaje automático que explica la relación entre el **sesgo** y la **varianza** de un modelo, y cómo afecta su capacidad de generalizar a datos nuevos. En esencia, el objetivo es encontrar un equilibrio entre **sesgo** (bias) y **varianza** para minimizar el error total.



El error total de un modelo (Error Total) se puede descomponer en tres componentes principales:

$$\text{Error Total} = \text{Bias}^2 + \text{Varianza} + \text{Error Irreducible}$$

Sesgo (Bias): Mide qué tan lejos están las predicciones promedio del modelo de los valores reales del conjunto de datos. Un alto sesgo indica que el modelo es **demasiado simple** y no captura los patrones subyacentes (subajuste).

$$\text{Bias}^2 = \left(\mathbb{E}[\hat{f}(x)] - f^*(x) \right)^2$$

La varianza mide cuánto varían las predicciones del modelo para diferentes conjuntos de datos de entrenamiento. Refleja la sensibilidad del modelo a cambios en los datos de entrenamiento.

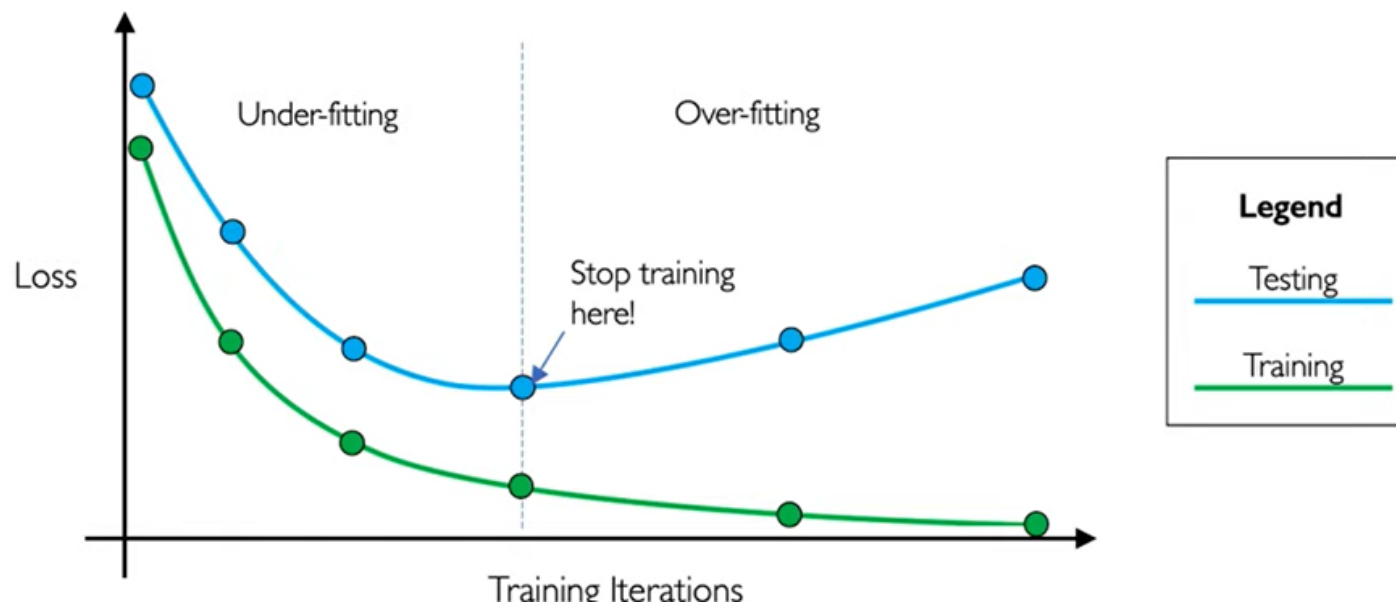
$$\text{Varianza} = \mathbb{E} \left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)] \right)^2 \right]$$

$\hat{f}(x)$ predicción del modelo para un punto x , $\mathbb{E}$ es la Esperanza o predicción promedio y $f^*(x)$ es el valor verdadero

El Error Irreducible es el ruido inherente en los datos que no puede ser eliminado ni explicado por ningún modelo, y permanece constante independientemente del modelo utilizado.

Aspecto	Overfitting (Sobreajuste)	Underfitting (Subajuste)
Sesgo	Bajo: Captura incluso patrones irrelevantes.	Alto: No capta patrones importantes.
Varianza	Alta: Muy sensible a los datos de entrenamiento.	Baja: Predicciones consistentes, pero incorrectas.

Una **curva de pérdida** (loss curve) es un gráfico que representa cómo cambia el valor de la función de pérdida del modelo durante el proceso de entrenamiento. Se utiliza principalmente para evaluar y diagnosticar el rendimiento del modelo de aprendizaje automático. La función de pérdida mide qué tan bien el modelo está ajustando los datos. Su objetivo es minimizar este valor durante el entrenamiento.

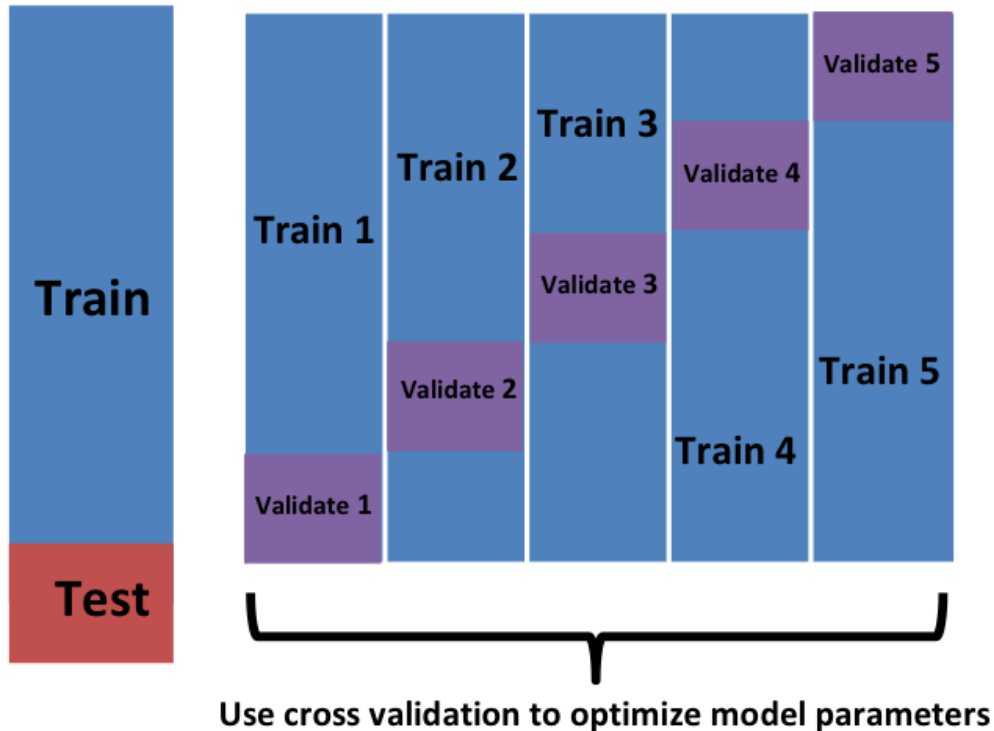


En un buen ajuste, las pérdidas de entrenamiento y validación disminuyen gradualmente y alcanzan valores similares, indicando una buena generalización. En el caso de sobreajuste, la pérdida de entrenamiento sigue disminuyendo mientras que la de validación comienza a aumentar, lo que muestra que el modelo memoriza los datos de entrenamiento pero no generaliza bien. Por último, en el subajuste, ambas pérdidas permanecen altas y no mejoran significativamente, lo que indica que el modelo es demasiado simple o insuficientemente entrenado.

Aspecto	Overfitting (Sobreajuste)	Underfitting (Subajuste)
Definición	El modelo es demasiado complejo y se adapta incluso al ruido de los datos.	El modelo es demasiado simple y no captura los patrones subyacentes.
Sesgo	Bajo: Captura patrones complejos, pero también ruido.	Alto: Incapaz de capturar patrones complejos.
Varianza	Alta: Predicciones inestables con diferentes conjuntos de entrenamiento.	Baja: Predicciones consistentes pero simplistas.
Rendimiento	Error bajo en entrenamiento. Error alto en prueba debido a falta de generalización.	Error alto en entrenamiento. Error alto en prueba debido a patrones no aprendidos.
Causas principales	Modelo excesivamente complejo (e.g., demasiada profundidad, demasiados parámetros). Uso insuficiente de regularización. Datos con ruido o outliers que afectan al modelo.	Modelo demasiado simple (e.g., lineal para datos no lineales). Regularización demasiado fuerte. Falta de características relevantes en el conjunto de datos.
Cómo detectarlo	Alta precisión en entrenamiento, pero baja en prueba. Diferencia grande entre error de entrenamiento y prueba.	Baja precisión tanto en entrenamiento como en prueba. Curvas de aprendizaje con error estabilizado en un nivel alto.
Soluciones	Aplicar regularización (e.g., L1, L2, dropout). Reducir la complejidad del modelo (e.g., limitar profundidad de árboles). Recolectar más datos para entrenamiento. Validación cruzada para evaluar el rendimiento generalizado.	Usar un modelo más complejo (e.g., pasar de lineal a no lineal). Añadir características relevantes (feature engineering). Reducir la regularización si está excesiva. Usar métodos avanzados como Boosting o redes neuronales profundas.

Validación cruzada

La **validación cruzada** es una técnica fundamental para evaluar el rendimiento del modelo y estimar su capacidad de generalización. Dividir el conjunto de datos en múltiples pliegues y entrenar/evaluar el modelo en diferentes combinaciones de pliegues permite obtener una estimación más fiable del rendimiento del modelo en datos no vistos.



```
# Perform crossvalidation
cv_scores = cross_val_score(model1, X, y, cv=5)

# Print the mean CV score
print("Mean CV Score:", np.mean(cv_scores))

# Interpretation
print("\nInterpretation:")
if np.mean(cv_scores) > 0.5:
    print("The mean crossvalidation score is above 0.5, indicating that the model performs better than random guessing.")
else:
    print("The mean crossvalidation score is below 0.5, indicating that the model does not perform better than random guessing.")
```

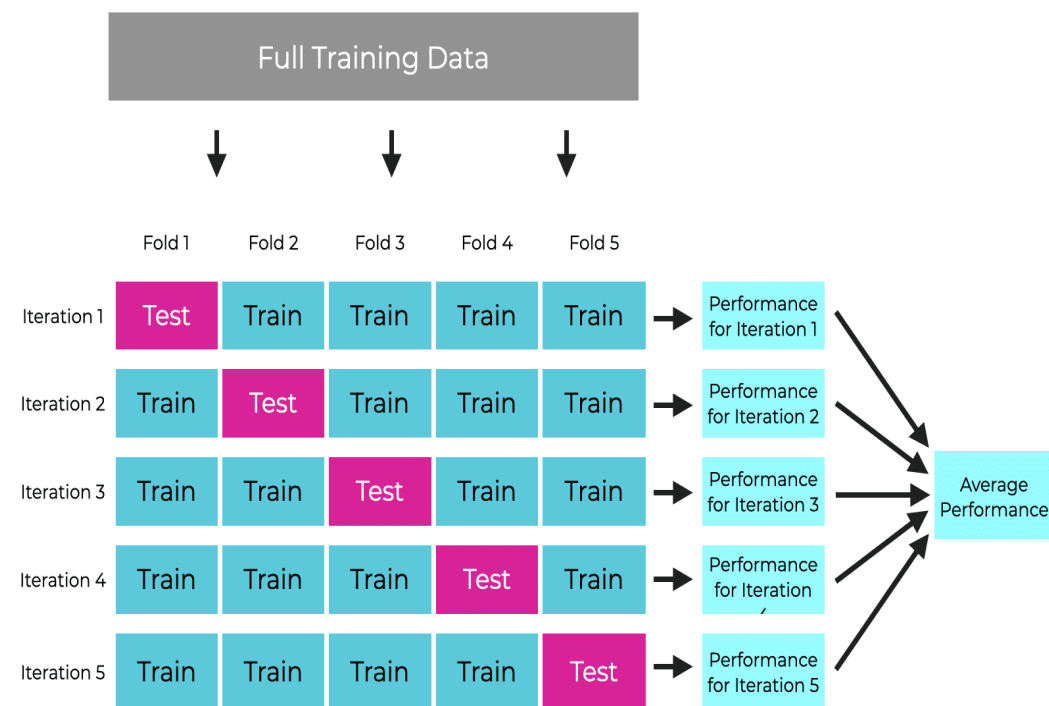
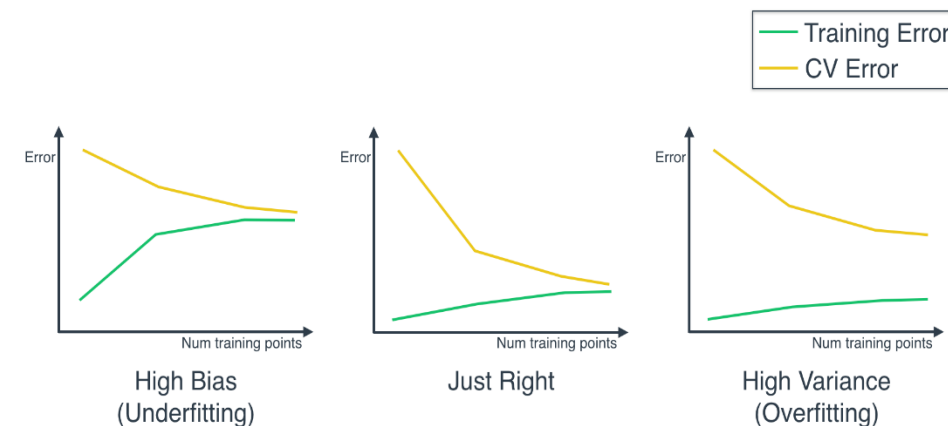
```
# Calcular el puntaje de entrenamiento
model1.fit(X_train, y_train)
train_score = model1.score(X_train, y_train)

# Calcular el puntaje de validación cruzada
cv_scores = cross_val_score(model1, X_train, y_train, cv=5)
cv_score_mean = np.mean(cv_scores)

# Comparación de puntajes
print("Puntaje de Entrenamiento:", train_score)
print("Puntaje de Validación Cruzada:", cv_score_mean)

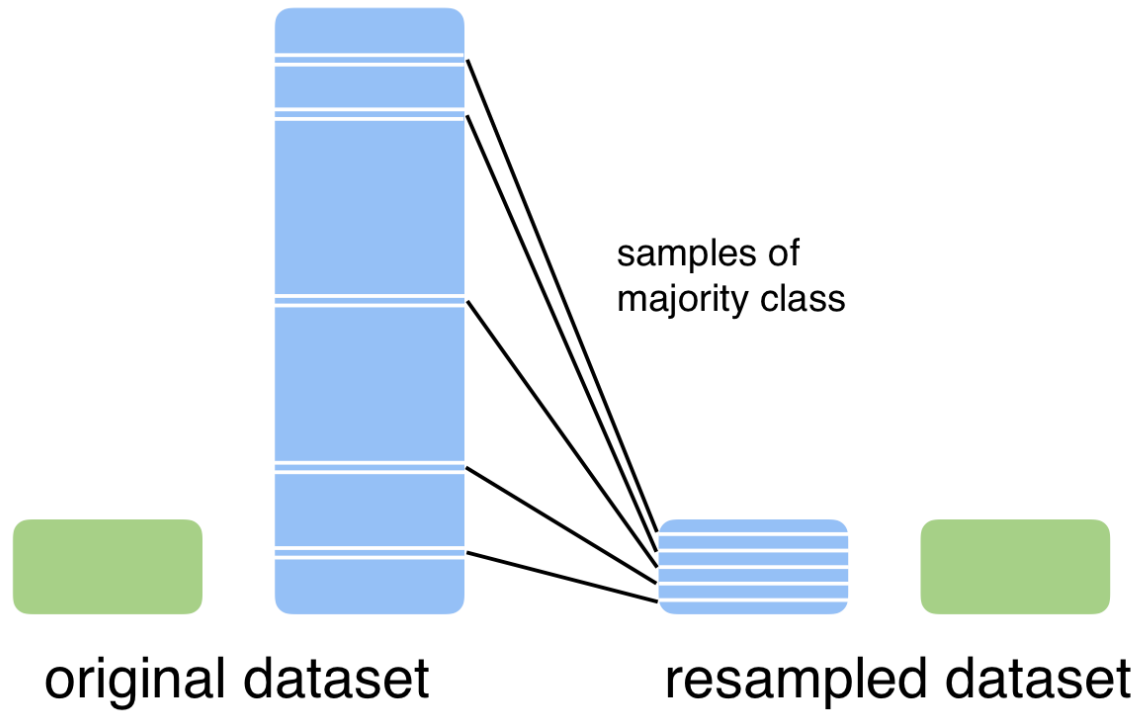
# Interpretación
if train_score > cv_score_mean*1.2:
    print("El modelo está sobreajustado, ya que tiene un rendimiento mucho mejor en los datos de entrenamiento que en los de validación cruzada.")
else:
    print("El modelo parece estar generalizando bien, ya que los puntajes de entrenamiento y validación cruzada son similares.")
```

Condición	Train (Score)	Cross (Score)	Indicador Principal	Conclusión
Bien Ajustado	Alto	Alto	Los puntajes de entrenamiento y validación son altos y cercanos entre sí. Ejemplo: Entrenamiento: 0.90, Validación: 0.88.	El modelo generaliza bien.
Overfitting	Muy Alto	Bajo o Significativamente Más Bajo	El modelo memoriza los datos de entrenamiento pero no generaliza bien a nuevos datos. Ejemplo: Entrenamiento: 1.0, Validación: 0.65	Ajustar hiperparámetros, aplicar regularización (L1/L2, Dropout). Reducir la complejidad del modelo (menos capas, menos árboles, menor profundidad).
Underfitting	Bajo	Bajo	El modelo no está aprendiendo correctamente ni en entrenamiento ni en validación. Ejemplo: Entrenamiento: 0.60, Validación: 0.58.	Incrementar complejidad del modelo, más épocas o ajustar datos.

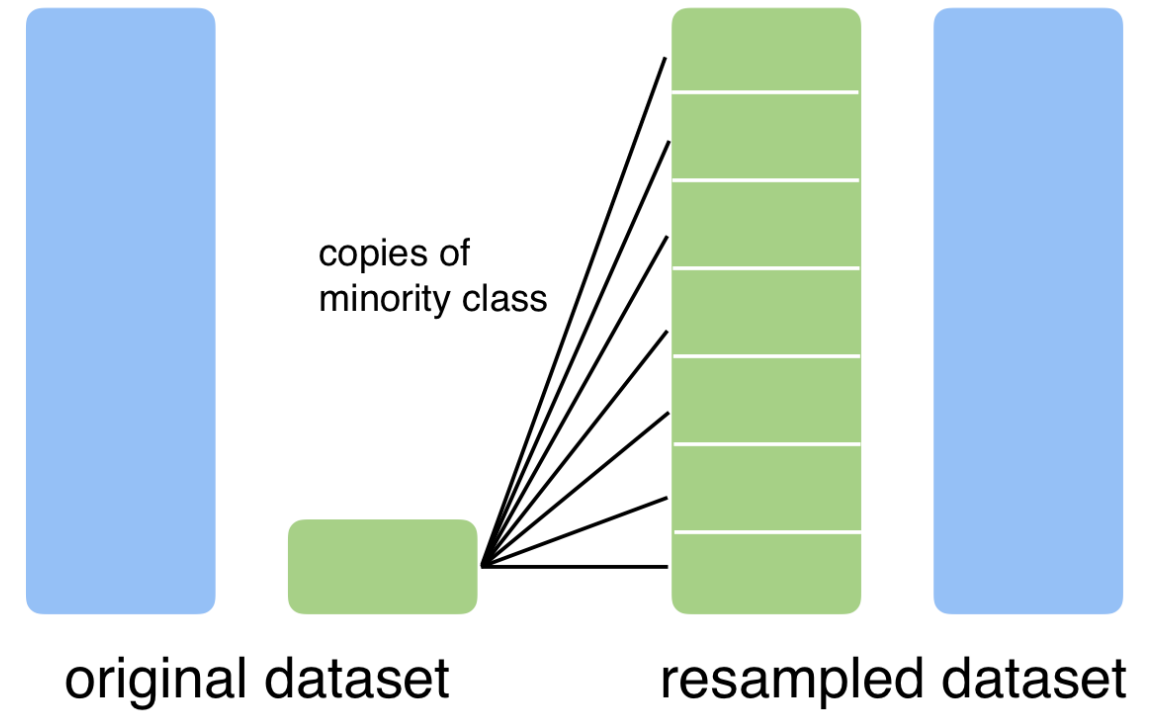


Técnicas de desequilibrio de clases

Undersampling



Oversampling



El sobremuestreo incluye métodos como **RandomOverSampler**, que replica muestras de la clase minoritaria, y SMOTE, que genera muestras sintéticas mediante interpolación. El submuestreo, por otro lado, reduce el tamaño de la clase mayoritaria con técnicas como **RandomUnderSampler** y **NearMiss**. Además, existen métodos combinados como **SMOTEENN** y **SMOTETomek**, que integran técnicas de sobremuestreo y submuestreo para mejorar la calidad del balance en los conjuntos de datos.

Método	Definición	Descripción
Random Oversampling	Replica muestras de la clase minoritaria.	Duplica aleatoriamente las muestras de la clase minoritaria para igualar el tamaño con la clase mayoritaria.
SMOTE (Synthetic Minority Oversampling Technique)	Genera muestras sintéticas para la clase minoritaria.	Crea nuevas instancias sintéticas interpolando entre puntos vecinos de la clase minoritaria.
Random Undersampling	Elimina muestras de la clase mayoritaria.	Reduce aleatoriamente el tamaño de la clase mayoritaria eliminando instancias hasta igualarla con la minoritaria.
NearMiss	Selecciona las muestras de la clase mayoritaria más cercanas a la clase minoritaria.	Reduce la clase mayoritaria seleccionando muestras basadas en proximidad a puntos de la clase minoritaria.
SMOTE + ENN	Combinación de SMOTE y Edited Nearest Neighbors (ENN).	Sobremuestrea la clase minoritaria con SMOTE y elimina instancias ruidosas cercanas a la frontera de decisión con ENN.
SMOTE + Tomek Links	Combinación de SMOTE y eliminación de Tomek Links.	Sobremuestrea la clase minoritaria con SMOTE y elimina pares de puntos ambiguos (Tomek Links) entre clases mayoritaria y minoritaria.


```
from imblearn.over_sampling import RandomOverSampler, SMOTE
from imblearn.under_sampling import RandomUnderSampler, NearMiss
from imblearn.combine import SMOTEENN, SMOTETomek

methods = {
    # RANDOM OVERSAMPLING
    "Random Oversampling": RandomOverSampler(
        sampling_strategy='auto' # Ajusta la proporción entre clases:
                                # 'auto': Igualar la clase minoritaria al tamaño de la mayoritaria.
                                # float (e.g., 0.5): Igualar la clase minoritaria al 50% del tamaño de la
mayoritaria.
    ),

    # SMOTE
    "SMOTE": SMOTE(
        sampling_strategy='auto', # Igual que en RandomOverSampler.
        k_neighbors=5,           # Número de vecinos cercanos usados para generar puntos sintéticos.
                                # Valores mayores permiten interpolar con más muestras, pero pueden incluir
ruido.
        random_state=42          # Fija la semilla para garantizar resultados reproducibles.
    ),

    # RANDOM UNDERSAMPLING
    "Random Undersampling": RandomUnderSampler(
        sampling_strategy='auto', # Ajusta la proporción entre clases:
                                # 'auto': Igualar la clase mayoritaria al tamaño de la minoritaria.
                                # float (e.g., 0.5): Igualar la clase mayoritaria al 50% del tamaño de la
minoritaria.
        random_state=42          # Fija la semilla para garantizar resultados reproducibles.
    ),
```

```
# NEARMISS
    "NearMiss": NearMiss(
        version=1,
        # Variante del algoritmo NearMiss:
        # 1: Selecciona los puntos de la clase mayoritaria más cercanos a cada punto
        # 2: Selecciona los puntos de la clase mayoritaria más alejados de la clase
        # 3: Promedia las distancias de los puntos mayoritarios para seleccionar.
        # Número de vecinos a considerar al calcular las distancias.
        # Igual que en RandomOverSampler.
        n_neighbors=3,
        sampling_strategy='auto'
    ),

# SMOTE + ENN
    "SMOTE + ENN": SMOTEENN(
        sampling_strategy='auto',
        random_state=42,
        smote=SMOTE(
            k_neighbors=5
        )
    ),

# SMOTE + TOMEK LINKS
    "SMOTE + Tomek Links": SMOTETomek(
        sampling_strategy='auto',
        random_state=42,
        smote=SMOTE(
            k_neighbors=5
        )
    )
}
```